Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 21:05:38	shalinisundarbalaji@gmail.(	Shalini M	shalinisundarbalaji@gmail.com	https://github.com/shalinisundarbalaji/RAG_Buildathon	1. build_vector_db.py combines a PyPDFLoader PDF with a WebBaseLoader page and tags web documents with source_type and source_url metadata before chunking at 1000/200. 2. The FAISS index is saved with save_local and app.py loads it once through st.cache_resource, so the Streamlit search does not re-embed documents on each run. 3. app.py uses similarity_search_with_score with k=3 and displays each chunk with its source and FAISS distance, which makes retrieval quality easy to inspect. 4. app.py rejects empty questions, handles empty results, and wraps index loading in try/except to show a clear error when agriculture_faiss_index is missing.	1. There is no LLM step, so app.py only lists raw chunks; add a prompt that injects the top chunks, answers only from them, refuses when unknown and cites sources. 2. The data is a generic FAO page and an Agriculture_Guide.pdf not in the repo; load Tamil Nadu agriculture scheme PDFs or tnagrisnet pages and commit them or a download script. 3. all-MiniLM-L6-v2 is English-only and cannot handle Tamil questions; switch to a multilingual model such as intfloat/multilingual-e5-base or BAAI/bge-m3 and rebuild the index. 4. agriculture_loader.py, load_urls.py and search_agriculture.py duplicate build_vector_db.py and app.py; delete them, add a requirements.txt, and move paths and model names into config. 5. There is no evaluation code; create ten questions with expected source chunks and measure hit-rate@3 and MRR, then add faithfulness scoring once generation exists.	Needs rework	**30**
14/10/2026 16:21:17	udhay.natrayan@gmail.com	Udhaya Sankar	udhay.natrayan@gmail.com	https://github.com/Udhaya-San/buildathon-tn-agriculture-rag	1. v11_streamlit_ui.py's prompt says to answer only from context and to reply with a fixed 'Information not available' sentence, which gives a clear refusal when retrieval finds nothing. 2. v11_streamlit_ui.py shows the source page numbers and an expander with each retrieved chunk under the answer, so users can check the answer against the PDF text. 3. v3_chunk_inspection.py and v4_pdf_validation.py print sample pages and chunks from the Tamil PDF to check extraction quality before embedding, which is a sensible debugging habit. 4. load_components in v11_streamlit_ui.py is wrapped in st.cache_resource, so the embeddings, FAISS index and LLM load once instead of on every Streamlit rerun.	1. No script calls FAISS.from_documents or save_local and faiss_index/ is gitignored, so the app cannot run from the repo; add a build_index.py that splits, embeds and saves the index. 2. The Knowledge Base metrics are hardcoded to 181 pages and 1100 chunks; compute them from the loaded index (vectorstore.index.ntotal) so the dashboard shows real numbers. 3. The v1 to v11 scripts repeat the same loading code; move ingestion, indexing and answering into modules with a config file and keep the Streamlit app as one entry point. 4. Retrieval is a plain k=3 similarity search on a single PDF; add MMR or BM25 hybrid search and attach scheme names as metadata so specific schemes are found more reliably. 5. There is no evaluation or chat history; add a set of test questions with expected pages to measure hit-rate, and keep previous turns in st.session_state for follow-up questions.	Needs rework	**36**
14/10/2026 22:56:29	nithyasree.sap@gmail.com	Nithyakalyani Udhay	nithyasree.sap@gmail.com	https://github.com/nudhayasurian/tamilnadu-agri-rag	1. src/crawler.py checks robots.txt with RobotFileParser and restricts links to the start host, so the crawl stays polite and confined to the official Tamil Nadu site. 2. src/downloader.py validates the '%PDF-' header, deduplicates files by sha256 hash and names them by content hash, which prevents corrupted or duplicate PDFs entering the corpus. 3. src/pdf_loader.py:extract_pdf uses PyMuPDF page by page and stores source, title, page and modality metadata, giving every chunk a traceable page reference. 4. backend/main.py exposes FastAPI health, stats, search and diagnostics endpoints with a lifespan-loaded FaissService, blank-query validation and proper 422, 503 and 500 error responses. 5. backend/services/faiss_service.py verifies the index metric and embedding dimension at startup and supports an optional max_distance cutoff to drop weak matches from results.	1. src/vector_store.py:build_index calls extract_pdf(pdf_path) before pdf_path exists, raising UnboundLocalError; delete that line and initialise documents as an empty list so indexing runs. 2. There is no LLM generation step, so this is semantic search rather than RAG; add a grounded prompt that answers only from retrieved chunks, refuses when unsure, and cites pages. 3. src/ocr_service.py:ocr_pdf_page is never called; wire it into extract_pdf for pages with no text so scanned Tamil scheme PDFs are actually indexed. 4. The splitter uses '।', a Devanagari danda, and the language filter in faiss_service.search is commented out; use Tamil-aware separators and store a language field to filter on. 5. There is no evaluation code beyond two splitter and loader unit tests; add a labelled query set and compute hit-rate@5 and MRR against the FastAPI search endpoint.	Needs rework	**37**
15/10/2026 01:41:00	mrjaiganesh24@gmail.com	jaiganesh	mrjaiganesh24@zohomail.in	https://github.com/mrjaiganesh24-spec/GenAi-Assign	1. App.py runs a complete Streamlit chat flow from PDF loading through FAISS retrieval to a gpt-4o-mini answer, with messages stored in st.session_state and replayed on each rerun. 2. initialize_vector_db is wrapped in st.cache_resource, so the PDF is embedded only once per server process instead of on every user interaction. 3. The prompt template tells the model to answer 'based only on the following context', and retrieved chunks are joined into that context before each LLM call. 4. Four preset question buttons cover micro-irrigation, solar pumpsets, organic farming and application steps, feeding the same RAG chain as typed questions.	1. initialize_vector_db re-embeds the PDF and overwrites vectorstore/faiss_index on every start but never loads it; call FAISS.load_local when the index exists and rebuild only when missing. 2. The prompt has no refusal rule and no sources are shown; add 'say you don't know if the context lacks the answer' and display page numbers from doc.metadata. 3. The header cards show hardcoded figures (100%, 70%, 50% subsidy) that are not retrieved from the PDF; remove them or generate them from indexed content. 4. Retrieval is plain similarity search with k=3 over 500-character chunks; add MMR or BM25 hybrid search and Tamil query translation for farmer questions. 5. There is no try/except around retrieval or the LLM call and python-dotenv is missing from requirements.txt; add error handling and complete the dependency list.	Basic	**40**
14/10/2026 13:33:38	titu.durais@gmail.com	Titu Duraisamy	titu.durais@gmail.com	https://github.com/DURAIT/Buildathon_RAG_App/tree/main	1. FastAPI_backend.py:ingest_data loads the URL with WebBaseLoader and falls back to Playwright's AsyncChromiumLoader when the page is empty or loading fails. 2. The /ingest endpoint accepts several uploaded PDFs, loads each with PyPDFLoader from a temporary file, and deletes the file in a finally block. 3. The system prompt tells gpt-4o-mini to answer strictly from the retrieved context and not use outside knowledge, and create_retrieval_chain injects the top five chunks. 4. FastAPI and Streamlit are separated cleanly: Streamlit_UI.py calls /ingest and /query over HTTP and shows clear errors when the backend is not running.	1. The deepeval import uses AnswerRelevanceMetric, but the class is named AnswerRelevancyMetric, so DEEPEVAL_AVAILABLE stays False; fix the name and run it over a fixed test set. 2. The FAISS index lives only in a global variable and is lost on restart; call save_local after ingestion and load_local on startup so documents are not re-embedded. 3. Chunks keep no scheme or department metadata and the answer returns no sources; add metadata during loading and return the cited document and page with each answer. 4. requirements.txt leaves out fastapi, uvicorn, streamlit, langchain-classic, deepeval and pypdf; list every package the two scripts import so a fresh install works. 5. The NeMo Guardrails block is just a pass statement and the UI sends the OpenAI key in every request; delete the dead block and read the key server-side.	Basic	**42**
14/10/2026 15:18:50	aslamtawfeeqayeesha@gm	Tawfeeq Ayesha	aslamtawfeeqayeesha@gmail.com	https://github.com/tawfeeqayesha/W3-D1-Buildathon-RAG-assignment-	1. Rag_app.py loads the API key from .env with load_dotenv and checks it before indexing or querying, so no secret is committed and missing keys give clear errors. 2. The prompt tells gpt-4o-mini at temperature 0 to answer only from the retrieved context and to state clearly when information is unavailable, which limits hallucination. 3. input_guardrail blocks queries containing terms like jailbreak or system prompt, and output_guardrail replaces empty or very short answers with a safe fallback message. 4. The query flow uses a clean LCEL chain of retriever, format_docs, prompt, LLM and StrOutputParser, wrapped in try/except with st.exception for visible errors. 5. Retrieved chunks are shown in expanders with one-based page numbers from PyPDFLoader metadata, so users can check the answer against the source page.	1. The app is a UAE Policy Intelligence Assistant whose default path 'KG_RAG_project/KG Rag doc.pdf' is not in the repo; re-target it to Tamil Nadu agriculture scheme documents and commit them. 2. The FAISS index lives only in st.session_state and is rebuilt every session; call save_local after building and load_local at startup to persist it. 3. The retriever runs twice per question, once for retrieved_docs and again inside the chain; pass retrieved_docs into the prompt directly so displayed sources match the answer context. 4. Retrieval is plain similarity with k=4 and there is no chat history; add MMR or BM25 hybrid search, keep st.chat_message history, and require inline page citations. 5. There is no requirements.txt and no evaluation; add a requirements file and a small question set scored with RAGAS faithfulness and hit-rate, saved to CSV.	Basic	**42**
14/10/2026 16:50:09	aslamtawfeeqayeesha@gm	Tawfeeq Ayesha	aslamtawfeeqayeesha@gmail.com	https://github.com/tawfeeqayesha/Task-Buildathon-RAG-assignment-	1. The query prompt in Rag_app.py tells the model to answer only from retrieved context and to say the information is unavailable when the context cannot answer. 2. Retrieved chunks are shown in expanders with corrected 1-based page numbers, so users can check the exact passages behind each answer. 3. Ingestion and querying are wrapped in try/except with st.exception, and the app checks for a missing API key, missing file path and unindexed state before running. 4. input_guardrail blocks queries containing jailbreak terms such as 'system prompt' or 'bypass', and output_guardrail replaces empty or very short answers with a safe fallback.	1. The app is a 'UAE Policy RAG Assistant' indexing a PDF path that is not in the repo; rebuild it on Tamil Nadu agriculture scheme sources committed to the repo. 2. FAISS is built in session_state and lost on restart; persist it with save_local in a separate ingestion script and load it with st.cache_resource in the app. 3. The retriever runs twice per query, once for display and once inside the chain; retrieve once, format those documents into the prompt and reuse them for the expanders. 4. Chunks only carry page metadata; add scheme name, department and source URL during ingestion, and include them in the context so the model can cite sources. 5. There is no requirements.txt, chat history or evaluation; add pinned dependencies, st.chat_message history and a 15-question test set scored for hit-rate and faithfulness.	Basic	**42**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
15/10/2026 01:24:25	venravi26@gmail.com	Ravikumar V	Ravikumar.venkatesan@yahoo.in	https://github.com/Ravi260878/TNAgriScheme	1. TNAgriScheme.py loads tnagrisnet and tnhorticulture pages with PlaywrightURLLoader and strips header, footer, nav, script and style elements before chunking. 2. The app grades retrieved context with an LLM YES/NO check and, on NO, rewrites the query and searches again, which is a working corrective-RAG loop. 3. The answer prompt allows only facts supported by the context, gives a fixed refusal sentence, and forbids inferring eligibility, deadlines or benefit amounts. 4. Each retrieved chunk is labelled with its source URL in the context, and the UI lists the unique source URLs as clickable links under the answer.	1. st.session_state.vector_store is only set inside the else branch, so a new session with an existing index raises AttributeError; assign it after loading in both branches. 2. FAISS_INDEX_PATH is hardcoded to C:\RAG_TN_AGRI\My_FAISS_TNAgri; read it from an environment variable or use a path relative to the script. 3. Only two landing pages are scraped; crawl the scheme detail pages and GO PDFs, and store scheme name, department and URL as chunk metadata. 4. There is no requirements.txt, no chat history and no try/except around LLM calls; add a requirements file, keep messages in session_state and handle API errors. 5. There is no evaluation code; build a ground-truth question set and log hit-rate@5 and faithfulness, also checking how often the rewrite step improves retrieval.	Basic	**44**
14/10/2026 13:10:19	sathish.varun@gmail.com	Sathish S	sathish.varun@gmail.com	https://github.com/sathishvarun/SE-B12/tree/main/Week2	1. Day4/rag_app.py loads multiple uploaded PDFs with PyPDFLoader, keeps the original filename and one-based page number in metadata, and cleans up temporary files in a finally block. 2. Day4/rag_app.py splits pages with RecursiveCharacterTextSplitter at 1000 characters and 150 overlap, and labels each retrieved chunk with [Source: file, Page: N] in the prompt context. 3. The Day4 system prompt forbids invented facts, asks the model to report conflicting values with file and page, and requires saying when documents lack enough information. 4. Day4 shows unique file and page combinations under the answer and wraps indexing and answering in try/except blocks that show readable Streamlit error messages. 5. Day5/app.py passes earlier turns through MessagesPlaceholder in an LCEL chain, so the grocery assistant remembers items across messages and keeps a running bill.	1. None of the code addresses the Tamil Nadu agriculture schemes task; build the assistant on scheme PDFs or tnagrisnet pages with scheme name, department and URL metadata. 2. Day4 keeps FAISS only in st.session_state, so every session re-embeds all uploads; save the index with save_local and load it on startup when it exists. 3. Day4 retrieval is a plain similarity retriever with k=6; add BM25 hybrid search or MMR plus Tamil-to-English query translation for farmer questions. 4. There is no evaluation code; add a ground-truth question set and measure hit-rate@5 and answer faithfulness, logging the results to a CSV file. 5. Day4 and Day5 have no requirements.txt and Day4 reads OPENAI_API_KEY without load_dotenv; add a requirements file per project and load configuration consistently from .env.	Basic	**45**
14/10/2026 23:26:33	mano.baskar@gmail.com	Baskar Manoharan	mano.baskar@gmail.com	https://github.com/manobaskar/New_assgnments/tree/e578af18c8c3c9f547c7d1844fdd3925416abd23/tn_scheme_rag	1. ingest.py and rag.py both use paraphrase-multilingual-MiniLM-L12-v2, so Tamil and English questions are embedded in the same space as the scraped scheme text. 2. ingest.py scrape_page loads the official tn.gov.in scheme list, strips script, style, nav and footer tags, and follows linked tn.gov.in pages, skipping pages under 200 characters. 3. store_chunks splits text with RecursiveCharacterTextSplitter at 800/150 and upserts chunks into a persistent Chroma collection with source URL metadata, so the index survives restarts. 4. rag.py ask_rag tells the model to answer only from context, return a fixed 'information is not available' message when unsupported, and reply in the same language as the question. 5. The whole stack runs locally with Chroma and Ollama, needing no API keys, and app.py keeps a Streamlit chat history and lists source URLs under each answer.	1. app.py imports from Github_assignments.New_assignments.tn_scheme_rag.rag, which does not exist in this repo, so the app cannot start; change it to 'from rag import ask_rag'. 2. Chunk IDs use abs(hash(source)), which Python randomises per process, so re-running ingest.py duplicates chunks; use hashlib.sha1(source.encode()).hexdigest() for stable IDs instead. 3. scrape_page follows every tn.gov.in link from the list page, not only agriculture schemes; filter links to scheme detail pages and store scheme_name and department as metadata. 4. venv311 and chroma_db are committed and there is no requirements.txt; add them to .gitignore, remove them from the repo, and list chromadb, sentence-transformers, ollama and streamlit with versions. 5. Retrieval is a plain Chroma query with n_results=5 and qwen2.5:0.5b is very weak; add BM25 hybrid search with a reranker and use at least a 3B–7B local model.	Basic	**45**
14/10/2026 15:20:40	nivi.bm13@gmail.com	Nivetha B	nivi.bm13@gmail.com	https://github.com/nivibm13-a11y/Agriculture	1. app.py:retrieve_documents merges semantic results from several queries with a keyword_score over the whole FAISS docstore, then prefers distinct source pages so context is not repetitive. 2. make_english_retrieval_query translates Tamil questions into English search queries with the LLM, while the original Tamil question is still used for answer generation. 3. generate_answer requires an immediate [SOURCE N] citation for every scheme fact and checks the output script, making one repair call if the answer comes back in the wrong language. 4. build_context labels each passage with document name, source type and one-based PDF page, and the UI shows these source cards plus an optional debug view with keyword scores. 5. analyze_crop_image validates image type and size, asks the vision model for visible features only, and feeds the observation into retrieval queries while banning it as scheme evidence.	1. load_vector_store expects a faiss_index built by ingest.py, but neither is in the repository; commit the ingestion and chunking script so the index can be rebuilt from source PDFs. 2. There is no requirements.txt, so dependencies like langchain-huggingface and faiss-cpu are undocumented; add a pinned requirements file and a .env.example listing OPENAI_API_KEY and OPENAI_MODEL. 3. The system prompt hardcodes facts like Rs 4,000 per acre (page 14) and Rs 648.55 crore (page 7); remove them so answers rely only on retrieved passages. 4. There is no evaluation code; add a 20-question English and Tamil test set measuring hit-rate@5 and faithfulness with RAGAS, and save the results to a CSV. 5. Everything sits in one 881-line app.py; split it into retrieval, prompting and UI modules, and move phrase lists and crop query templates into a config file.	Basic	**46**
15/10/2026 20:59:44	jackmani.mech@gmail.com	Manikandan	jackmani.mech@gmail.com	https://github.com/MsMani-AI/GEN-AI---Social-Eagle-/tree/main/WEEK%202	1. day 4/rag_app.py replaces the temporary upload path with the real file name and converts PyPDFLoader's zero-based page to a one-based page before splitting, so citations match the PDF. 2. The PROMPT in rag_app.py requires the exact reply 'Not found in the uploaded documents.' when excerpts lack the answer, and a [file name - page N] citation after each fact. 3. The prompt also tells the model to report conflicting numbers or rules across documents with each source instead of picking one, which is uncommon and useful grounding behavior. 4. rag_app.py stores the FAISS index in st.session_state and rebuilds it only when the set of uploaded file names changes, avoiding repeated embedding on each rerun.	1. The app is a generic multi-PDF uploader without a TN agriculture corpus; add a fixed folder of TN scheme PDFs with scheme and department metadata and persist the index. 2. There is no requirements file and the model name is hardcoded; add requirements.txt and read the model, chunk size and TOP_K from environment variables. 3. all-MiniLM-L6-v2 is English-only; switch to a multilingual embedding model such as paraphrase-multilingual-MiniLM or bge-m3 so Tamil scheme text and questions can be retrieved. 4. Questions go through a single st.text_input with no conversation memory; use st.chat_input with session history and add MMR or a reranker on top of the k=4 retriever. 5. There is no evaluation; add a set of TN scheme questions with expected file and page, and measure hit-rate@4 and faithfulness, saving the results to a CSV.	Basic	**46**
15/10/2026 07:38:40	mgaayathri27@gmail.com	Gaayathri Muralidhar	mgaayathri27@gmail.com	https://github.com/Gaayathri96/Buildathon_1/tree/main/tn-agri-raghttps://github.com/Gaayathri96/Buildathon_1/tree/main/tn-agri-rag	1. ingest.py reads four Tamil Nadu agriculture policy PDFs with PyMuPDF, page by page, and keeps source path and page number as metadata on every chunk. 2. read_pdfs_and_scrape_urls also follows hyperlinks embedded in each PDF page, strips script and style tags, and appends that web text to enrich the page content. 3. app.py's system prompt restricts answers to the provided context and returns a fixed refusal sentence when the information is not in the loaded documents. 4. load_rag_pipeline is wrapped in st.cache_resource, so the persisted Chroma store, retriever and chain are built once instead of on every Streamlit rerun.	1. There is no requirements.txt and app.py imports the non-standard langchain_classic package; add pinned dependencies including langchain-classic, pymupdf, chromadb and beautifulsoup4. 2. rag_chain.invoke has no try/except, so API or retrieval failures crash the page; wrap it and show a friendly st.error message with the failure detail. 3. Scraped web text is merged into the PDF page and tagged with the PDF's page number, so citations are misleading; store scraped pages as separate documents with their URL. 4. Retrieval is plain similarity search with k=4; add MMR or BM25 fusion, and a chat_message interface with history so users can ask follow-up questions. 5. chroma_db and __pycache__ are committed and there is no evaluation; gitignore them and add a labelled question set with hit-rate@4 scoring saved to a CSV.	Basic	**49**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 18:24:27	sairamanathanvs@gmail.co	SAIRAMANATHAN V	sairamanathanvs@gmail.com	https://github.com/vssairam/TN_AgriBOT	1. answer_question in TN_AgriBOT_Assistant.py labels each retrieved chunk with filename and page, and the system prompt demands context-only answers with [Source: file, page] citations. 2. build_faiss_index writes each upload to a unique temp file, overrides source with the original filename, converts pages to 1-based numbers, and always deletes the temp file afterwards. 3. render_sources deduplicates retrieved chunks and shows each one in a filename-and-page expander, letting farmers verify the exact passage behind every answer. 4. The Streamlit app reads OPENAI_API_KEY only from the environment, disables building when it is missing, and catches build and answer errors with clear user-facing messages. 5. The sidebar offers build, replace, reset and clear-chat controls, and chat history persists in session_state with sources stored alongside each assistant message.	1. The FAISS index lives only in st.session_state and is rebuilt per session; persist it with save_local and reload it with load_local on startup instead of re-embedding. 2. Two TN scheme PDFs sit in the repo root but are never loaded automatically; add an ingestion script that indexes them with scheme and department metadata by default. 3. Retrieval is plain similarity_search with k=5; switch to MMR or combine BM25 with FAISS so near-duplicate pages do not crowd out other relevant scheme chunks. 4. There is no evaluation code; add a small set of scheme questions with expected pages and measure hit-rate@5 plus answer faithfulness with RAGAS or an LLM judge. 5. Everything lives in one 330-line Streamlit file with hardcoded model names; split ingestion, retrieval and UI into modules and read models and chunk sizes from environment variables.	Basic	**50**
14/10/2026 22:56:52	heyram4ru@gmail.com	Udhaya Surian Thang	heyram4ru@gmail.com	https://github.com/Uthangavel060382/Tamil-Nadu-Agriculture-RAG-Model	1. ingestion/crawl_sources.py:crawl limits crawling to the seed domains, retries temporary HTTP errors, caps page and PDF counts, and writes a crawl_report.json with errors. 2. ingestion/ingest.py:ingest_text builds SHA-256 chunk IDs from text and metadata, so running ingestion again updates existing Chroma entries instead of adding duplicates. 3. backend/embeddings.py uses paraphrase-multilingual-MiniLM-L12-v2 with normalised vectors, so Tamil and English queries share one embedding space for scheme search. 4. crawl_sources.py extracts financial_year from source config or URL patterns and stores it with page numbers, document type and source URL on every chunk. 5. tests/test_application.py patches in stub embeddings and a temporary database to test the /search API, repeated ingestion, PDF page numbering and the 503 error path.	1. No LLM is called anywhere, so this is semantic search and not RAG; pass the top chunks to an LLM with an 'answer only from context' prompt and cite sources. 2. backend/rag.py:search_schemes uses plain similarity_search_with_score only; add MMR or a BM25 hybrid, plus a reranker, to improve precision on scheme-name queries. 3. The tests check API plumbing but not answer quality; add a labelled question set and measure retrieval hit-rate and MRR against the crawled corpus. 4. The UI shows a 'department' field, but no ingestion code ever sets department metadata; add department to sources.py and pass it through the crawl metadata. 5. The root chroma_db is an empty file and root app.py only smoke-tests the crawler; delete both and add one documented command that crawls and then starts the backend.	Basic	**50**
14/10/2026 18:04:10	kesavanelevenyears@gmai	Kesavan	kesavanelevenyears@gmail.com	https://github.com/KesavanAPJ/buildathon	1. get_compressed_retriever wires BM25Retriever (k=10) into a ContextualCompressionRetriever with a BAAI/bge-reranker-base CrossEncoderReranker that keeps the top 3, so reranking is actually used for every question. 2. SYSTEM_PROMPT tells Gemini to answer only from retrieved context, to say when the document lacks the information, and to cite pages as [PDF page X] using the supplied labels. 3. Each chunk gets a one-based page_label, and the document_prompt puts that label in front of the chunk text, so page citations in answers match real PDF pages. 4. The app checks that the upload is non-empty and starts with the %PDF header, drops empty chunks, and shows readable errors with a technical-details expander when indexing or answering fails.	1. Retrieval is BM25-only with no embeddings, so paraphrased or Tamil questions miss relevant chunks; add a multilingual dense index and combine it with BM25 using an EnsembleRetriever. 2. The app handles one uploaded PDF and rebuilds the index each session; load a fixed corpus of TN scheme PDFs from a folder and save the index to disk. 3. There is no requirements.txt even though the code needs langchain_classic, langchain_google_genai and sentence-transformers; add a pinned requirements file so the app installs reliably. 4. Each question is answered on its own with no conversation memory; switch to st.chat_input with session history and rewrite follow-up questions before retrieval. 5. There is no evaluation; add a test question set and compare hit-rate and faithfulness for BM25 alone against BM25 plus the reranker, saving the scores to a CSV.	Basic	**52**
14/10/2026 22:16:36	arulmurugan@live.in	Arul Murugan	arulmurugan@live.in	https://github.com/arul2589/RAG-Buildathon	1. app.py:scrape_once renders the TNAU page with Playwright, extracts only the State Level Schemes section, refuses suspiciously short text and saves scrape metadata with a UTC timestamp. 2. TNAURAG._ensure_local_index stores embeddings in a SQLite table and re-embeds only when the SHA-256 of the source text changes, so restarts do not repeat embedding calls. 3. The system prompt in TNAURAG.__init__ restricts answers to the supplied excerpts, forbids guessing current policy and tells the model to say it could not find missing answers. 4. The FastAPI service exposes /ask with a validated Pydantic request model and a /health endpoint, and returns 503 while the RAG object is still loading.	1. The module-level st.App(__file__, routes=[Mount(...)]) runs on every import including uvicorn app:api; remove it or guard it, since most Streamlit versions have no st.App attribute. 2. The corpus is one TNAU paddy page section with one static source URL; ingest more scheme pages and store scheme name and section metadata per chunk for real citations. 3. TNAURAG.ask loads every vector from SQLite and computes cosine similarity in a Python loop; use FAISS or Chroma, which requirements.txt already lists, and add MMR or BM25 hybrid. 4. The Streamlit form keeps no history and shows only one generic source link; use st.chat_input with session_state messages and display the retrieved excerpts under each answer. 5. There is no evaluation code; write 15 paddy-scheme questions with expected answers and score retrieval hit-rate@4 and answer correctness with an LLM judge, saving results to a CSV.	Basic	**52**
14/10/2026 23:49:29	ashokramachandran.it@gm	R Ashok kumar	rashokkumar.it@gmail.com	https://github.com/ashokramachandranit:png/Buildathon_RAG_Chatbox_AGRICULTURE	1. scraper.py scrape_tn_agri_schemes uses Playwright on seven real TN portals (aed.tn.gov.in, TNAU, GRAINS, PMFBY) and wraps each URL in try/except, so one bad site does not stop scraping. 2. rag_chain.py builds a Role-Task-Context-Format-Rules system prompt that limits answers to retrieved portal data and tells the model to say the information is unavailable when it is missing. 3. query_rag_with_sources returns the unique source URLs from the retrieved chunks, and Agri_app.py shows them in an Official Sources expander under every answer. 4. Agri_app.py transcribes microphone input with Whisper and reads answers aloud with gTTS, choosing Tamil or English by checking for Tamil Unicode characters, which helps farmers who prefer speaking. 5. clean_text_content strips YouTube prompts, social-media links and footer text with regex before chunking, which keeps boilerplate noise out of the FAISS index.	1. rag_chain.py imports langchain_classic, but requirements.txt does not list it or python-dotenv, so a fresh install fails; add both packages and pin their versions. 2. query_rag_with_sources calls get_rag_chain on every question, reloading FAISS each time; cache the chain with st.cache_resource and pass session messages into chat_history, which is always empty. 3. Retrieval is plain as_retriever with k=4; add MMR or a BM25 ensemble retriever with a reranker so scheme-name queries and paraphrased questions both find the right chunk. 4. There is no evaluation code; write a script with about 20 scheme questions and expected source URLs and record hit-rate@4 and answer faithfulness to a CSV. 5. Chunks carry only source and type metadata from index pages; scrape the linked scheme sub-pages and add scheme_name and department so retrieval can filter by scheme.	Basic	**54**
14/10/2026 23:58:03	mohanbakthadu@gmail.co	Mohan Sekar Bakthad	mohanbakthadu@gmail.com	https://github.com/mohanbakthadu-jpg/BUILDATHON	1. Web_Scrap-app.py:scrape_website uses Playwright to render the official TNAGRISNET GoScheme portal, follows only /people_app/ links on that host, and waits 0.5 s between pages. 2. Each scraped page becomes a Document whose text starts with its title and source URL and whose metadata also holds the URL, so retrieved chunks are traceable to the portal. 3. create_vectorstore saves the FAISS index with save_local, and the app restores it at startup with load_saved_vectorstore, so users do not need to re-scrape every session. 4. answer_question's prompt says to answer only from the retrieved context, never invent amounts or eligibility rules, and say clearly when the indexed pages cannot verify the answer. 5. The Streamlit chat shows a 'View retrieved source passages' expander with the title, URL and full text of each passage, and catches scraping and answering errors with clear messages.	1. RAG_UI.py and RAG_n_UI.py are generic NetApp PDF demos next to the agriculture app; remove them and split Web_Scrap-app.py into scraper, indexer and UI modules. 2. all-MiniLM-L6-v2 is an English-only embedding model, so Tamil questions will retrieve poorly; switch to paraphrase-multilingual-MiniLM-L12-v2 or OpenAI text-embedding-3-small. 3. Retrieval is a plain similarity retriever with k=4; add MMR or BM25 hybrid search, and pass recent chat history so follow-up questions can be rewritten before searching. 4. playwright is missing from requirements.txt and MAX_PAGES caps the crawl at 10 pages; add the package, make the limit configurable, and remove the committed electoral-roll PDF. 5. There is no evaluation code; write 15–20 scheme questions with expected source URLs and compute hit-rate@4 plus an LLM-judge faithfulness score, saving the results to CSV.	Basic	**54**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 14:26:38	jaisreee2021@gmail.com	Jaisree	jaisreee2021@gmail.com	https://github.com/jaisreemoss-cell/Buildathon_TN_RAG/tree/main	1. tn_rag_app.py split_by_scheme uses a regex on scheme headings to make one chunk per scheme, and enforce_max_size splits only chunks over 2,000 characters, which is deliberate structure-aware chunking. 2. Every chunk gets state, source URL, chunk_id and heading metadata, and the heading is shown in the Scheme sections used expander so users see which scheme was used. 3. scraper.py extract_tamil_nadu_section keeps only the text between the Tamil Nadu and Kerala headings and raises a clear error if either marker disappears. 4. load_vector_store is cached with st.cache_resource and index_is_stale rebuilds FAISS only when scraped_schemes.txt is newer than the index, so normal runs load from disk. 5. The Streamlit chat keeps session history, streams answers with st.write_stream, offers example questions, a top-k slider and a rebuild button, and catches API errors with a readable message.	1. The whole knowledge base is one TNAU paddy page of about 12 KB; add aed.tn.gov.in, tnagrisnet and scheme PDFs, each tagged with its own source and department. 2. Retrieval is plain similarity_search; use MMR or combine BM25 with FAISS so exact scheme names like NFSM and broader questions both return the right chunks. 3. There is no evaluation script; build 20 questions with the expected scheme heading and measure hit-rate@k and answer faithfulness, saving results to a CSV. 4. Sources are shown as the first 80 characters of each chunk; also show the source URL from metadata so farmers can check the official page. 5. Everything, including about 100 lines of CSS, lives in tn_rag_app.py and nothing handles Tamil; split into modules and support Tamil questions and Tamil answers.	Basic	**57**
14/10/2026 22:10:29	guru4ui@gmail.com	Kumara guru	guru4ui@gmail.com	https://github.com/kumaraguruj/B12-Buildathan-RAG-Agricultural-Schemes-Assistant-	1. aggri_assist.py:get_vectorstore compares a fingerprint of PDF names, sizes and modified times with pdf_fingerprint.txt, and rebuilds the FAISS index only when the PDFs change. 2. SYSTEM_PROMPT in aggri_assist.py covers context-only answers, out-of-scope refusals and not-covered replies, uses Benefits/Eligibility/How to apply headings and requires [file p.N] citations. 3. aggri_assist.py:add_to_history and load_history save every question, answer and its sources to search_history.json, and the sidebar lets users reopen or clear past searches. 4. The chat flow in aggri_assist.py catches errors from answer() and shows them in the chat, and stops early with clear messages when the API key or PDFs are missing.	1. answer() sends only the current question to the LLM, so follow-up questions lose context; pass recent st.session_state.messages and rewrite follow-ups into standalone queries before retrieval. 2. Retrieval is a plain k=4 similarity retriever; add MMR or BM25 hybrid search plus a reranker, and tag chunks with scheme names so they can be filtered. 3. There is no evaluation code; add a script with 20 or more test questions and expected pages that reports hit-rate@4 and LLM-judged faithfulness and saves the results. 4. All 417 lines, including roughly 150 lines of CSS, sit in one file; split indexing, QA, history and styling into modules and read model and chunk settings from env variables. 5. faiss_index and search_history.json are committed, requirement.txt omits python-dotenv, which the code imports, and pyproject requires Python 3.14 or newer; gitignore those files and fix the dependencies.	Basic	**57**
14/10/2026 23:52:26	sathees0103@gmail.com	Satheeshkumar Nag	sathees0103@gmail.com	https://github.com/sath0103/genai/tree/main/RAG	1. ingestloadvectoredb.load_pdf_documents loops over every PDF in data/, skips empty pages, records filename and 1-based page metadata, and catches errors per file. 2. The FastAPI lifespan handler loads the persisted FAISS index and LLM once at startup and fails fast with clear messages if the API key or index is missing. 3. ask_question labels each context chunk with its source file and page, tells the model to use only that context, and returns deduplicated file and page sources. 4. run_streamlit calls the /ask API and handles connection errors, timeouts and HTTP errors separately, shows a backend health check, and keeps chat history with source expanders. 5. Paths, model name, host and port are read from environment variables through dotenv, and .gitignore excludes .env and the faiss_index folder from the repo.	1. The knowledge base is a single generated guide PDF; scrape official tnagrisnet or Uzhavan scheme pages and GO PDFs, and add scheme and department metadata to each document. 2. Retrieval is plain similarity search with k=4; add BM25 plus FAISS fusion or MMR, and a distance threshold so weak matches trigger the refusal message. 3. The Streamlit history is only displayed, never sent to /ask; pass the last few turns and rewrite follow-up questions into standalone queries before retrieval. 4. There is no evaluation code; create a 20-question ground-truth set over the scheme guide and measure hit-rate@4 and answer faithfulness with RAGAS, logging results to a CSV. 5. app.py mixes FastAPI and Streamlit code, and its error message tells users to run ingest.py, which does not exist; split the modules and reference ingestloadvectoredb.py.	Basic	**57**
14/10/2026 23:52:26	priazpage@gmail.com	Priyatharshini Govind	priazpage@gmail.com	https://github.com/prijiturise/buildathon	1. farmer_chatbot.py's system prompt restricts scheme facts to the retrieved context, forbids inventing amounts or deadlines, requires disagreements to be cited by file and page, and answers in Tamil when appropriate. 2. farmer_chatbot.py tags every context passage with its filename and 1-based page, and stores the deduplicated sources with 700-character excerpts in session state, so earlier answers keep their citations. 3. rag_app.py builds the FAISS index into a temporary folder and swaps it in with a backup restore on failure, so a crashed rebuild never destroys the working vector store. 4. rag_app.py exposes chunk size and overlap sliders, checks that overlap is smaller than size, and reports PDFs that yield no text as possibly needing OCR. 5. The chatbot wraps retrieval and generation in try/except, checks for the API key and vector store before starting, and offers Tamil and English suggested questions.	1. Retrieval is store.as_retriever with plain similarity at k=6; add BM25 through an EnsembleRetriever or MMR plus a reranker so exact scheme names and varied wording both match. 2. Chunks carry only source filename and page; add scheme name and department metadata during indexing so answers can be filtered and cited by scheme, not only by PDF page. 3. index.faiss and index.pkl are committed at the repo root while the app loads scheme_vectorstore/, so a fresh clone fails; commit them under that folder or document the indexing step. 4. download_goscheme.py imports Playwright, which is missing from requirements.txt, and its output PDF is not indexed automatically; add the dependency and connect its output to rag_app.py. 5. There is no evaluation code; write a script with 20 labelled scheme questions that records whether the correct file and page appear in the top 6 and scores faithfulness.	Basic	**58**
14/10/2026 22:43:45	srikanth.ai1221@gmail.com	Srikanth Jayaram	srikanth.ai1221@gmail.com	https://github.com/srikanthai1221/agri-rag	1. config.py keeps every source list, the Chroma directory and the 1000/150 chunk settings in one place, so ingest.py and app.py stay free of hardcoded values. 2. ingest.py loads PDFs with PyPDFLoader, static pages with WebBaseLoader and JavaScript pages with AsyncChromiumLoader plus Html2Text, tagging each document with source_type and loader metadata. 3. app.py turns the sidebar Search in choice into a Chroma source_type filter and lets users set k with a slider, so the metadata filter is actually used in retrieval. 4. source_label shows the PDF file name with page number, or the web URL, for every retrieved chunk, and the answers are stored in session history with their sources. 5. The prompt says to answer ONLY from the context and to say the information is missing otherwise, and app.py returns a fallback message when nothing is retrieved.	1. config.py points at Wikipedia pages, the TNAU homepage, a national annual report and a 1964 PDF; replace these with Tamil Nadu scheme sources such as aed.tn.gov.in and tnagrisnet. 2. Chunks carry only source_type, source and loader metadata; add scheme_name and department during ingestion so users can filter by scheme rather than file type. 3. No evaluation code exists; add a script with 20 ground-truth scheme questions that measures hit-rate@k and RAGAS faithfulness and writes the results to a CSV. 4. retriever.invoke and chain.stream in app.py have no try/except, so a network or API error crashes the page; catch errors and show a friendly st.error message. 5. Retrieval is plain dense search and nothing handles Tamil; add a BM25 ensemble or reranker and a multilingual embedding or translation step so Tamil questions work.	Good	**60**
14/10/2026 22:56:22	heyram4ru@gmail.com	Udhaya Surian Thang	heyram4ru@gmail.com	https://github.com/Uthangavel060382/tamilnadu-agri-RAG	1. backend/app/ingest.py:crawl follows only scheme-like same-host links from seed_urls.json, and fetch_pdf checks the %PDF magic bytes and size cap to reject soft-404 HTML pages. 2. backend/app/ingest.py:chunk_text cuts 800-character chunks with 120 overlap at line or sentence boundaries and drops fragments under 80 characters, keeping source, title, kind and domain metadata. 3. config.py uses intfloat/multilingual-e5-base, which handles Tamil, and vectorstore.py:_embed correctly applies the 'query: ' and 'passage: ' prefixes the e5 model expects. 4. schemes.py:index_schemes turns structured JSON records into scheme cards with jurisdiction and category metadata, and vectorstore.search filters them with a Chroma where-clause on kind. 5. frontend.py:garbled_tamil detects broken Tamil text from legacy-font PDFs and flags it as unreliable, and the FastAPI /search, /schemes and /health endpoints are cleanly separated.	1. There is no LLM or answer generation anywhere; pass the top-k /search passages to an LLM with an answer-only-from-context prompt, a 'not found' refusal and [source] citations. 2. Retrieval is a single dense Chroma query; add BM25 hybrid search with reciprocal rank fusion or a cross-encoder reranker so exact scheme names and abbreviations rank reliably. 3. The 20 MB venv/, __pycache__ files and the Chroma database are committed; add a .gitignore for them and rebuild the index from the ingest script instead. 4. There is no evaluation code; create a mixed Tamil and English question set with expected sources and log hit-rate@5 and MRR from vectorstore.search to a CSV. 5. read_pdf uses pypdf, which garbles legacy-font Tamil PDFs; add PyMuPDF or OCR fallback for pages that garbled_tamil flags so those chunks become searchable.	Good	**60**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 12:54:57	premkumarbazskarraj@gm	Premkumar Bazskar	premkumarbazskarraj@gmail.com	https://github.com/Prembazskar/Buildthon_Use_Case_RAG-	1. pipeline.py:extract_pdfs collects every link and URL in the official scheme PDFs, and scrape_url fetches those portal pages, so referenced web content is indexed alongside the PDFs. 2. Every Document carries source_file, page, source_url and content_type metadata, and scrape_url strips script, nav, header and footer tags and skips non-HTML or very short pages. 3. TN_AgriApp.py drops results with FAISS L2 distance of 1.0 or more and returns a fixed 'not enough information' answer when nothing relevant remains. 4. The prompt numbers each chunk as [Source N], requires answers only from context with citations, and treats retrieved text as evidence rather than instructions. 5. The 'View retrieved sources' expander shows file, PDF page, source URL, L2 distance and an excerpt for each cited chunk, so users can check every answer themselves.	1. st.session_state.messages is displayed but never sent to the LLM, so follow-up questions lose context; pass the last few turns and rewrite the question before searching. 2. The fixed L2 threshold of 1.0 is untuned and may drop valid matches; calibrate it on sample questions or always keep the top two results. 3. Retrieval is plain similarity_search_with_score with k=5; add BM25 hybrid fusion or MMR so exact scheme names and acronyms rank reliably. 4. There is no evaluation code; build a 20-question ground-truth set from the two PDFs and record hit-rate@5 and LLM-judge faithfulness in a CSV. 5. The app says to run 'python ingest.py' but the script is pipeline.py; fix the message and move hardcoded paths and models into an environment-based config module.	Good	61
14/10/2026 17:38:56	padmalojani@gmail.com	Padmalojani K	padmalojani@gmail.com	https://github.com/padmalojani/AgricultureSubsidySchemeRAGAppln	1. The project is split into config.py, ingestion.py, vector_store.py, prompts.py and utils.py, with models and LangSmith settings read from environment variables instead of being hardcoded. 2. ingestion.py:process_uploaded_pdf reads pages with PyMuPDF, stores source, page, file_hash and scheme_name metadata, and refuses fully scanned PDFs that need OCR. 3. vector_store.py persists the FAISS index and add_documents_to_store adds new uploads to it incrementally, so earlier documents are not re-embedded. 4. prompts.py:QA_PROMPT tells the model to use only the documents, never invent amounts or deadlines, report conflicts, and passes chat_history through a MessagesPlaceholder for follow-up questions. 5. app.py adds a filename and page source list from utils.format_sources under every answer, and tests/test_rag.py checks hashing, file validation and source formatting.	1. rag_chain.py's format_docs joins page_content only, so the model never sees filenames or pages; label each chunk '[file, p. N]' so the prompt's per-claim citation rule can work. 2. Retrieval is plain similarity_search with k=5; add MMR or BM25 hybrid fusion, and use a score threshold so weak matches trigger the refusal message. 3. data/ is gitignored and no data is shipped or downloaded, so the app is empty until someone uploads PDFs; add a script that fetches the official TN scheme PDFs. 4. file_hash is computed but never used to skip duplicates, so re-uploading a PDF adds the same chunks again; check the hash against existing metadata before indexing. 5. There is no retrieval or answer evaluation; add a ground-truth question set and measure hit-rate@5 and faithfulness with RAGAS or an LLM judge, logging results to CSV.	Good	61
14/10/2026 13:50:58	jaichandran.net@gmail.com	Jaichandran	jaichandran.net@gmail.com	https://github.com/Chandrujai/Buildathon	1. WebscrapeWithModal.py:scrape uses Playwright to page through the tnagrisnet GoScheme table and opens each scheme modal, saving eligibility, required documents and subsidy patterns to CSV. 2. app.py:extract_csv and extract_xlsx turn each row into labelled 'column: value' text with row-level location metadata, which suits structured scheme tables better than raw CSV text. 3. app.py:split_text is a custom splitter that breaks at newline or sentence boundaries, with CHUNK_SIZE and CHUNK_OVERLAP read from environment variables and validated before use. 4. app.py:index_file uses a SHA-256 file hash with a Chroma PersistentClient, so re-uploading the same file is skipped and the index survives app restarts. 5. answer_question instructs the model to use only supplied context, say when the answer is missing, cite filenames and locations, and treat instructions inside documents as data.	1. app.py:retrieve is plain cosine top-k; add BM25 hybrid search or MMR and use Chroma where-filters on Department or Scheme columns to narrow results. 2. There is no evaluation code; write a script with 20 ground-truth scheme questions that measures retrieval hit-rate@5 and answer faithfulness, and saves results to CSV. 3. The scrapers are not connected to app.py, so the scraped CSV must be uploaded by hand; add an ingest step that indexes output/tn_agriculture_go_schemes.csv automatically. 4. app.py is a generic 'Local Persistent RAG' with no agriculture or Tamil handling; add Tamil query translation and an agriculture-scheme system prompt that answers in the user's language. 5. WebscrapeWithModal.py launches Chromium with headless=False and convert_csv_to_pdf references a commented-out PDF_FILE; run headless by default and remove or fix the dead PDF code.	Good	62
14/10/2026 16:05:42	vviyas@gmail.com	Vedhaviyas Ramesh	vviyas@gmail.com	https://github.com/VedhaRamesh/SE-AI-B12-2-W3-D1-Building-Knowledge-Graph-RAG	1. extract_schemes.py and extract_all_schemes.py:extract_scheme scrape the tn.gov.in scheme list and each detail table into scheme_details.csv, capturing department, funding, beneficiaries, benefits and source URL. 2. import_schemes_to_neo4j.py models the data as a real graph with MERGE on Department-OFFERS-Scheme plus Beneficiary and Benefit nodes, so repeated imports stay idempotent. 3. rag_answer.py builds a per-scheme context block and a strict prompt that forbids cross-scheme leakage, writes 'Not specified' for missing fields and requires the source URL for every scheme. 4. app.py:retrieve_schemes routes crop questions through the Cypher RELATED_TO_CROP traversal in get_schemes_for_crop and labels the retrieval method shown to the user in each reply. 5. app.py keeps chat history in st.session_state.messages, renders HTML-escaped scheme cards with official links, and catches exceptions with a friendly message instead of a stack trace.	1. RELATED_TO_CROP edges come only from query_graph.create_crop_relationships, which no script calls; run it at the end of import_schemes_to_neo4j.py so crop retrieval works after a fresh import. 2. Eligibility is scraped but import_schemes_to_neo4j.py never stores it, so answers always say 'Not specified'; add s.eligibility in the MERGE and return it from both Cypher queries. 3. The keyword fallback passes the whole lowercased question to a CONTAINS match, which rarely hits; add a Neo4j vector index on scheme descriptions and combine it with the graph lookup. 4. detect_crop only recognises four crops and get_search_keyword only three words; extract entities with the LLM or a lookup table built from scheme names to cover paddy, sugarcane and others. 5. validate_schemes.py only counts populated CSV fields; add a question set with expected schemes and measure retrieval recall and answer faithfulness, and add a requirements.txt.	Good	62
15/10/2026 09:26:49	prasanthspbio@gmail.com	Prasanth Sundararaj	prasanthspbio@gmail.com	https://github.com/prasanth-sundararajan/TNAU-Agri-Schemes-AI-Assistant.git	1. parse_schemes_from_html builds separate documents from tables, heading sections and the full page body, each tagged with scheme_title and type, so scheme details stay together. 2. extract_media_and_text_from_pdf collects PDF hyperlinks and sends embedded images to GPT-4o-mini vision for captions, so subsidy charts become searchable text. 3. rerank_documents is called in the live query path and uses an LLM to reorder the eight candidates and keep the top five before they reach the prompt. 4. The Streamlit app automatically crawls the TNAU portal when the Chroma collection is empty, and lists the indexed schemes as sidebar buttons that ask about each scheme. 5. Answers show the scheme title and source URL in a Referenced Sources expander, so farmers can check where each answer came from.	1. hybrid_retrieve puts eight semantic hits first, so combined[:k] drops every BM25 result; fuse the two lists with reciprocal rank fusion or LangChain's EnsembleRetriever. 2. MULTILINGUAL_RAG_PROMPT never says to answer only from the context or to admit missing information; add a strict grounding rule and a fixed 'not found' reply. 3. requirements.txt is missing streamlit, langchain-chroma, langchain-openai, langchain-community, rank_bm25 and python-dotenv; add every import so the app installs and runs. 4. ingest_documents adds chunks with no IDs, so uploading the same PDF twice creates duplicates; generate deterministic hash IDs from the source, page and content. 5. There is no evaluation code; build a set of labelled TNAU scheme questions and score retrieval hit-rate and answer faithfulness, saving the results to CSV.	Good	62
16/10/2026 22:12:50	kovarthanancse@gmail.cor	kovarthanan	kovarthanancse@gmail.com	https://github.com/kovarthanan/application-for-agri-scheme-chat/tree/master	1. scraper/tn_gov.py walks tn.gov.in department listings with Playwright, retries failed page loads in goto, downloads scheme PDFs and saves JSON after every scheme so runs can resume. 2. app.py:build_index attaches title, source URL, department and benefit_type metadata, and prefixes each chunk with the scheme name so split pieces of long schemes stay identifiable. 3. app.py:get_chain prompts the model to answer only from context, reply 'I don't know based on the scheme data' when missing, and answer in the question's language. 4. The Streamlit chat streams tokens with st.write_stream, keeps history in session_state and shows a 'Schemes used' expander with clickable tn.gov.in source links for every answer. 5. The index is persisted with save_local, built in batches with a progress bar, and reloaded through load_index, so the knowledge base is not re-embedded on every run.	1. Retrieval is a plain store.similarity_search(question, k=top_k); switch to MMR and add metadata filters on benefit_type or department, or combine BM25 with FAISS for hybrid search. 2. nomic-embed-text is English-centric although the UI invites Tamil questions; use a multilingual embedding model such as bge-m3 or multilingual-e5 for Tamil query support. 3. The repository has no requirements.txt; add one pinning streamlit, langchain-ollama, langchain-community, faiss-cpu, playwright and pypdf so others can reproduce the environment. 4. app.py mixes CSS, data loading, indexing, prompting and UI in one 450-line file; split it into ingest, retriever, chain and ui modules with settings read from environment variables. 5. There is no evaluation code; create 20 scheme questions with expected scheme titles, measure retrieval hit-rate@k and log answer faithfulness with an LLM judge to a CSV.	Good	62

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
15/10/2026 06:30:28	saragpd@gmail.com	Saravanan Sekar	saragpd@gmail.com	https://github.com/saragpd/Week_2_RAG_Buildathon	1. ingest.py crawls four official sites (tnagrisnet, TNAU, TN horticulture) with Playwright, follows keyword-matched same-domain links and extracts linked PDFs with pypdf. 2. The crawler writes sources.jsonl and an ingestion_manifest.json with page and PDF counts, and skips re-crawling unless --rebuild is passed, giving reproducible provenance. 3. build_index.py splits the corpus by source block before chunking, so every chunk keeps its source URL and title, and the FAISS index is saved and reused. 4. api.py's system prompt allows only retrieved context, forbids inventing amounts or deadlines, answers in Tamil or English, and the endpoint returns deduplicated title, URL and excerpt sources. 5. Model name, embedding model, retriever k and API URL come from environment variables, and the Streamlit app shows a clear message when the FastAPI backend is unreachable.	1. api.ask uses plain similarity search with k=4 over noisy crawled pages; add BM25 plus FAISS fusion or MMR, and a reranker to push scheme-specific chunks above boilerplate. 2. Chunks only carry source and title; extract scheme name and department during ingestion and store them as metadata so retrieval can filter or group by scheme. 3. The splitter separator '။' is a Myanmar punctuation mark that never appears in Tamil text; remove it or use Tamil-aware sentence boundaries such as newline and full stop. 4. app.py sends only the latest question to /ask; include recent chat turns in the request and rewrite follow-ups into standalone queries before retrieval. 5. There is no evaluation code; build a question set from the crawled schemes and measure hit-rate@4 and RAGAS faithfulness, saving results alongside the ingestion manifest.	Good	**64**
15/10/2026 01:49:10	svignesh899@gmail.com	Vignesh S	svignesh899@gmail.com	https://github.com/Vignesh2017/SE-AI-RAG-BOT-BUILDATHON	1. app.py builds an EnsembleRetriever combining BM25 (0.4) and FAISS (0.6) at k=15 each, then reranks with FlashRank ms-marco-MiniLM through ContextualCompressionRetriever, all wired into the query path. 2. build_vector_store_from_documents persists the FAISS index, a pickled BM25 retriever and a metadata.json, and load_saved_index reloads all three at startup so nothing is rebuilt on every run. 3. load_pdf_documents stores the filename and 1-based page_number on every page, and the answer renders citation cards with source, page and a 320-character excerpt that are saved in chat history. 4. SYSTEM_PROMPT says to answer ONLY from the supplied excerpts, never invent subsidy amounts, and flag missing details, and each excerpt is labelled with source and page. 5. Answers stream token by token with st.write_stream, and an optional NeMo Guardrails topical rail with a Tamil refusal message blocks off-topic questions when the package is installed.	1. There is no requirements.txt for langchain_classic, flashrank, pypdf and nemoguardrails; add pinned versions and split the 489-line app.py into ingestion, retrieval and UI modules. 2. SYSTEM_PROMPT line 133 reads 'Cite supporting evidence inline using.', which is truncated; finish it with the exact format, for example '[Excerpt n]', so inline citations are consistent. 3. The fingerprint is saved in metadata.json but never compared with current_fingerprint, so new PDFs never trigger a reindex; compare them at startup and rebuild when they differ. 4. load_pdf_documents swallows every exception with 'except: continue', and a FlashRank failure silently falls back to the ensemble; log these failures and show them in the sidebar. 5. There is no evaluation code; create a question set over the six bundled PDFs and compare hit-rate and MRR for BM25, FAISS, ensemble and reranked retrieval.	Good	**65**
14/10/2026 12:54:00	rajeshin06@gmail.com	Rajesh VIKRAMAN	rajeshin06@gmail.com	https://github.com/rajeshvyp/genai_Buildathon.git	1. scrapers/process_documents.py extract_pdf combines layout-aware text, PyMuPDF find_tables extraction and eng+tam Tesseract OCR fallback, choosing per page whichever output has real Tamil characters. 2. suspicious_tamil_text in process_documents.py detects mojibake markers like 'à®' and replacement characters, then sends those pages to OCR instead of indexing garbled Tamil text. 3. chatbot/rag_chatbot.py get_search_queries translates Tamil questions to English with the LLM and retrieve_documents searches both, merging duplicates by best distance, so Tamil queries reach English documents. 4. The system prompt in rag_chatbot.py restricts answers to the retrieved passages, forbids merging different schemes' conditions, asks the model to report conflicts, and treats passages as untrusted data. 5. build_vectorstore.py refuses to overwrite an existing FAISS index unless --rebuild is passed, and rag_chatbot.py caps context at MAX_CONTEXT_CHARS, protecting both API cost and prompt size.	1. process_documents.py writes to data/processed_review but build_vectorstore.py reads data/processed; point both at the same folder so a fresh run of the pipeline works end to end. 2. extract_page_marker expects '--- Page N ---', but process_documents writes '--- Page N | method=... ---', so page numbers are lost; update the regex to allow the method suffix. 3. chatbot/app.py ignores result['sources'] that ask_question returns; show source file, page and URL in an expander under each answer so users can check citations without trusting the LLM text. 4. There is no requirements.txt, and Tesseract is hardcoded to a Windows C:\ path; add pinned requirements and read TESSERACT_CMD from .env so the project installs on other machines. 5. There is no evaluation code; build a labelled set of English and Tamil scheme questions and measure hit-rate@12 and answer faithfulness, comparing results with and without bilingual retrieval.	Good	**67**
14/10/2026 13:01:46	pandeeswari.learning@gma	Pandeeswari Palpan	pandeeswari.learning@gmail.com	https://github.com/Pandeeswari-Palpandi/agri-mithran/tree/main	1. The backend is cleanly split into config.py, schemas.py, rag_service.py, guardrails.py and api.py, and FastAPI loads RagService once through a lifespan handler. 2. rag_service.py _retrieve_relevant_documents asks gpt-4o-mini to grade each of the top-4 FAISS chunks YES/NO, so irrelevant chunks are filtered out before the answer is generated. 3. The answer prompt says to answer strictly from context and returns pydantic-validated JSON with scheme_name, funding_pattern, eligibility and how_to_apply, written in Tamil or English as requested. 4. RagService.search runs an LLM judge for faithfulness and answer relevance on every query, adds a retrieval relevance percentage, and the Streamlit expander shows a PASS/FAIL verdict. 5. app.py handles HTTPError and connection failures from the /api/search call separately and gives the user clear messages, including the uvicorn command needed to start the backend.	1. index_builder.py embeds with the English-only all-MiniLM-L6-v2 model although answers default to Tamil; switch to a multilingual model such as multilingual-e5-base or bge-m3. 2. load_documents calls raise_for_status inside the URL loop, so one failed site stops the whole build; wrap each URL in try/except and add scheme_name and department metadata. 3. When the grader marks every chunk NO, _retrieve_relevant_documents still returns the first two chunks; return a clear refusal instead so off-topic questions are not answered from weak context. 4. Evaluation runs only per query; add an offline set of about 20 questions with expected sources and save hit-rate, MRR and judge scores to a CSV for comparison. 5. Retrieval is plain dense top-4 search and the Streamlit page has a single text_input without history; add BM25 hybrid or reranking and switch to st.chat_input with session_state history.	Good	**67**
14/10/2026 14:58:47	prabathsmp@gmail.com	Prabath M	prabathsmp@gmail.com	https://github.com/prabathsmp/AI-B12-2/tree/main/Buildathon	1. download_uzhavan_schemes.py calls the Uzhavan portal's getSchemeDet endpoint per scheme and captures department, beneficiaries, documents and bilingual subsidy rows into per-scheme PDFs. 2. The scraper writes download_report.csv with scheme ID, department, source URL and success or failure status, so failed portal fetches are visible instead of silently dropped. 3. load_or_build_databases compares a size and mtime manifest of the PDFs and reloads the saved FAISS index when nothing changed, avoiding needless re-embedding on every start. 4. Before retrieval, rewrite_prompt uses the last ten chat messages to turn follow-up questions into standalone search queries, and the answer chain also receives that chat history. 5. SYSTEM_MESSAGE forces a verbatim refusal when context lacks the answer, and the app hides citations whenever the reply matches one of the REFUSAL_OR_FALLBACK_PHRASES.	1. retrieve_with_bm25_fallback only uses BM25 when no FAISS hit passes the 1.35 threshold; fuse both result lists with reciprocal rank fusion or EnsembleRetriever on every query. 2. extract_all_chunks keeps only source filename and page on chunks; add scheme name, department and scheme ID from download_report.csv so answers and filters can use them. 3. The context passed to the LLM joins only page_content, so the model cannot cite sources; prefix each chunk with its PDF name and page and require inline citations. 4. There is no evaluation code; add a 20-question ground-truth set over the Uzhavan schemes and measure hit-rate@4 and answer faithfulness, saving results to a CSV. 5. openapi-vivasai-app.py and ollama-vivasai-app.py duplicate about 860 lines differing only in get_llm; move retrieval, prompts and UI into shared modules and select the LLM through config.	Good	**67**
14/10/2026 20:29:48	anandr2k7@gmail.com	Anand	anandr2k7@gmail.com	https://github.com/Anand2904/RAG_TN_Agri	1. config.py defaults to the paraphrase-multilingual-MiniLM-L12-v2 embedding model with normalised vectors, so Tamil and English questions share one semantic space for retrieval. 2. rag_engine.py:answer_question builds numbered [Source n] context blocks with title, URL and department, and the prompt forbids invented subsidies, eligibility or deadlines and admits missing facts. 3. vector_store.py:build_vector_store builds into a temporary folder, test-loads it, swaps it in with a backup, and writes metadata.json with chunk count and indexing time. 4. document_processor.py strips scripts, navigation and footers from HTML, extracts PDF text with pypdf, and attaches url, department, government_level and category metadata to each document. 5. app.py offers an English/Tamil interface via translations.py, chat history, suggested questions, Ollama-specific error messages, and a scheme explorer filtered by level, category and department.	1. scraper.py fetches only the three landing pages in sources.json and committed metadata.json shows 1 document and 2 chunks; follow scheme and PDF links to build a real corpus. 2. Retrieval is plain db.similarity_search with TOP_K; add MMR or metadata filters on department and category, which are already stored, to diversify and target results. 3. tests only mock the LLM and check translation keys; add a ground-truth question set and compute retrieval hit-rate@k and answer faithfulness, saving results to a file. 4. Most modules are written as dense one-line statements separated by semicolons; reformat with black and split long expressions so the pipeline is readable and reviewable. 5. The committed scrape_manifest.json lists a crop_insurance.html entry not in sources.json and not produced by scraper.py naming; regenerate it from a real scraper run for reproducibility.	Good	**67**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 12:58:10	vilvakumarp@gmail.com	Vilvakumar P	vilva@yamail.com	https://github.com/vilvakumarp/SE-AI-B12_Buildathon_Oct2026	1. '1 - extract_agrisnet.py' walks all four cascading AGRISNET dropdowns with Playwright and records department, scheme, input type, component and source URL, logging failed combinations. 2. document_loader.load_documents turns each scraped record into labelled text and keeps scheme, department, input_type, component and record_id as chunk metadata for display and deduplication. 3. rag_pipeline.answer_question abstains with a language-specific fallback when the best FAISS distance exceeds MAX_FAISS_DISTANCE, instead of letting the LLM guess from weak evidence. 4. guardrails.citations_valid checks that every [Source N] marker matches a supplied source, and strip_invalid_citations removes fake ones and adds an unverified warning to the answer. 5. The same answer_question is served by both main_app.py's chat with history and diagnostics and by api.py's typed FastAPI /ask endpoint, which also passes the conversation history.	1. _query_variants only adds hard-coded keyword expansions for pumps, machinery, documents and eligibility; replace them with BM25 plus dense fusion or LLM query rewriting for any topic. 2. The extractor writes to ./agrisnet_data, but document_loader and scheme_catalog read data/raw/agrisnet_schemes.json; align these paths and remove the hardcoded D:\ path in the validator. 3. main_app.py sends the language as 'தமிழ்' while _fallback checks for 'Tamil', so Tamil users get English abstentions; map the selectbox labels to the internal language codes. 4. There is no evaluation code; build a labelled question set and measure hit-rate@8, citation accuracy and abstention correctness against the distance threshold, saving results to a CSV. 5. playwright is used by the extractor but missing from requirements.txt, and filenames such as '2 build_knowledge_base.py' cannot be imported; add the dependency and rename the scripts.	Good	**69**
15/10/2026 11:22:52	contact.kapildevmahalinga	kapildev mahalingan	contact.kapildevmahalingam@gmail.com	https://github.com/kapildevmahalingam-ai/TamilNadu-Agri-RAG	1. api.py exposes /chat and a streaming /chat/stream endpoint with FastAPI, and app.py consumes the NDJSON stream, so the backend and the Streamlit UI are cleanly separated. 2. rag.py:rewrite_question uses Gemini to turn follow-up questions into standalone search queries from the last six messages before FAISS retrieval, which fixes ambiguous references. 3. app.py stores multiple conversations in chats.json with new and delete buttons, and shows the source PDF and page under each answer, so chats survive restarts. 4. eval_deepeval.py runs DeepEval Faithfulness, AnswerRelevancy and ContextualRelevancy metrics on questions from eval_questions.csv, giving an LLM-judged quality check of the pipeline. 5. rag.py:_messages tells the model to use only the supplied context, to say when nothing was found and to cite claims as [PDF filename, p. N].	1. ingest.py and rag.py use all-MiniLM-L6-v2, which is English-only, while the UI invites Tamil questions; switch to a multilingual model such as bge-m3 and rebuild the index. 2. eval_questions.csv has four questions, no expected answers, and N defaults to 3; add 20 or more questions with reference answers and save the metric scores to a CSV. 3. guardrails.py only checks for empty or overly long text, and guardrails_config holds just a README; add scope checks, prompt-injection checks and a check that citations match retrieved pages. 4. CHAT_MODEL defaults to 'gemini-3.8-flash', which looks like an invalid name; set a known model name as the default and fail clearly at startup if it is not available. 5. Remove the 'ingest - Temp_backup.py' and 'rag - temp_backup.py' files and the 19 MB screen recording from the repo, and keep chunk settings in an environment-driven config.	Good	**69**
14/10/2026 13:11:05	prithviraj11493@gmail.com	Prithivirajan R	prithviraj11493@gmail.com	https://github.com/Prithvi11J/Daily-Tasks/tree/main/RAG%20Buildathon	1. build_faiss_index.py combines four sources (readable PDF pages, saved web text, manually verified transcripts and OCR pages) and tags each one with source_type, page and source URL metadata. 2. build_faiss_index.py:load_selected_ocr_pages discards OCR pages with too little text or too few English words, which keeps garbled table text out of the index. 3. app.py:retrieve_documents offers MMR search (fetch_k at least 50, lambda 0.55) and filters results by the source types the user selects, so answers come from varied, chosen sources. 4. app.py:translate_for_search turns Tamil questions into English search queries before retrieval, and answer_language_instruction answers in Tamil when the user asks in Tamil. 5. The system prompt in app.py:display_answer requires [S#] citations, keeps figures unchanged and tells the model not to double-count Grand total rows, which targets a real error in scheme tables.	1. app.py and build_faiss_index.py use data/faiss_index but the repo ships Data/, which breaks on Linux and macOS; use one folder name and make the path configurable. 2. test_retrieval.py prints the results of one hardcoded query; replace it with a ground-truth question set that computes hit-rate@k and faithfulness and writes the scores to a file. 3. OCR_PAGES_TO_INDEX and the micro-irrigation file names are hardcoded, and ocr_preview.py points to a Windows Tesseract folder; move these to a config file so new PDFs can be added. 4. display_answer sends each question on its own with no chat history; keep the conversation in st.session_state and rewrite follow-up questions into standalone queries before retrieval. 5. Retrieval has no keyword component; add a BM25 retriever fused with the FAISS results so exact scheme names and GO numbers are found even when the embedding misses them.	Good	**71**
14/10/2026 15:59:37	pdineshkumar55@gmail.co	Dineshkumar Palanis	pdineshkumar55@gmail.com	https://github.com/pdineshkumar55/tn-agri-schemes-rag	1. scraper.py:parse_scheme_details reads the tn.gov.in scheme tables into labelled fields such as Beneficiaries and How To avail, giving clean structured text for 54 schemes. 2. build_index.py:split_documents adds the scheme name to the start of every chunk and keeps url and department metadata, so split chunks still say which scheme they cover. 3. rag_chain.py:build_rag_chain rewrites follow-up questions with condense_prompt only when there is chat history, so the first question skips the extra LLM call. 4. ANSWER_SYSTEM_PROMPT limits answers to the context, gives a fixed 'couldn't find' sentence, and requires naming the scheme; app.py hides sources when that sentence appears. 5. app.py streams tokens with st.write_stream, sends the last three turns as history, and shows the rewritten 'Searched for' query next to each answer.	1. Retrieval is plain top-4 similarity; add MMR or a BM25 hybrid over the scheme chunks, then a cross-encoder reranker, so keyword-heavy questions find the right scheme. 2. Evaluation is limited to one sanity_check query and a three-question demo; add a labelled question set and score hit-rate@4 and faithfulness, saving the results to CSV. 3. The pipeline is English-only with English OpenAI prompts; detect Tamil input, translate it for retrieval, and answer back in Tamil so local farmers can use it. 4. config.py raises an error on import when OPENAI_API_KEY is missing, which breaks even scraper.py; check for the key only where an OpenAI client is created. 5. scraper.py has no retry for failed scheme pages and silently drops them; add retries with backoff and record failed URLs so the dataset can be completed later.	Good	**71**
14/10/2026 16:15:57	raaj2012kumars@gmail.co	S Raajkumar	raaj2012kumars@gmail.com	https://github.com/Raajkuma/Advance_RAG.git	1. retrieval/hybrid_search.py HybridRetriever.search runs FAISS dense and BM25 searches in parallel threads and fuses them with reciprocal rank fusion (k=60), and app.py calls it for every query. 2. indexing/vector_index.py uses the multilingual BAAI/bge-m3 model with normalised inner-product FAISS, persisting dense.faiss and chunks.jsonl and skipping chunk_ids that are already indexed. 3. ingestion/ocr_engine.py extract_page_text uses native PDF text first, falls back to Tesseract tam+eng OCR, then to Gemini vision when quality is low, recording the extraction method per page. 4. ingestion/pdf_extractor.py writes a manifest entry for every page with sha256, method, quality estimate and status, and keeps failed pages in the audit trail instead of silently dropping them. 5. config.py reads models, chunk size, overlap, retrieval_k and directories from env, and app.py shows step-by-step ingestion and query progress with a chat history and per-answer sources.	1. generate_answer tells the model to answer 'primarily' from context; switch to the stricter, unused SYSTEM_PROMPT in generation/prompts.py, which requires [S1]-style citations and uses only the evidence. 2. retrieval/reranker.py and the configured bge-reranker-v2-m3 are never called; rerank the top RRF candidates down to RERANK_TOP_N before generation to sharpen the context. 3. evaluation/metrics.py defines recall_at_k and reciprocal_rank but nothing calls them; add a labelled question set with relevant chunk_ids and a script that logs both metrics. 4. ingestion/chunker.py finds semantic breakpoints with English-only all-MiniLM-L6-v2, so Tamil boundaries are unreliable; reuse bge-m3 for sentence similarity and add Tamil sentence-ending punctuation. 5. tests/test_chunker.py calls chunk_document(size=, overlap=), which do not exist, and google.generativeai is missing from requirements.txt; fix both and stop committing query_cache.db.	Good	**71**
14/10/2026 19:36:40	poovizhimangai@gmail.cor	Poovizhi Mangai Pat	poovizhimangai@gmail.com	https://github.com/poovizhimangaipatchirajan/Buildathon-Project/	1. backend/rag/retrieval.py:search_documents fuses FAISS dense results and BM25Retriever keyword results with reciprocal rank fusion, so exact scheme names and paraphrased questions both reach the context. 2. build_index.py saves the FAISS index and a separate bm25_chunks.pkl, so the keyword retriever uses the same chunks as the dense index without re-reading the PDFs. 3. RAG_PROMPT_TEMPLATE in chatbot.py tells the model to answer ONLY from the context, answer in English and Tamil, and use a fixed bilingual refusal when the answer is missing. 4. guardrails/validcheck.py:validate_input blocks prompt-injection patterns and off-topic questions without agriculture keywords before /chat and /chat-stream call the RAG pipeline. 5. evaluation/evaluate.py:run_evaluation checks whether the expected source was retrieved, scores context relevance, faithfulness and answer relevance with a GPT-4o-mini judge, and saves evaluation_report.csv.	1. TEST_DATASET has only two questions, and their expected PDFs (welfareschemes.pdf, agri_e_pn_2026_27.pdf) are not in data/pdfs; write 20+ questions against the current PDFs and rerun. 2. search_documents reloads FAISS and rebuilds BM25Retriever on every query; load both once at FastAPI startup (a lifespan handler or module cache) and reuse them for every request. 3. PyPDFLoader chunks keep only source and page metadata and the context drops even those; add scheme and department metadata and label each chunk so the model can cite page numbers. 4. data/users.xlsx commits plaintext usernames and passwords that main.py:verify_user reads; remove it from git, hash passwords with bcrypt, and load users from an ignored file or database. 5. requirements.txt is missing pandas, openpyxl, reportlab and rank_bm25 (needed by BM25Retriever), and BACKEND_BASE_URL is hardcoded; add the packages and read the URL from an environment variable.	Good	**71**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 23:20:03	simplers0710@gmail.com	surya	simplers0710@gmail.com	https://github.com/Surya-90/agriculture_self_rag_langsmith	1. rag_pipeline.py:answer_question runs a real Self-RAG loop: grade_relevance, rewrite_query and re-retrieve on weak results, then grade_grounding swaps in a safer answer if claims are unsupported. 2. generate_answer labels each chunk as [SOURCE n: file, page], tells the model to use only that context, cite sources and say when the documents lack enough information. 3. make_client wraps the OpenAI client with wrap_openai and every pipeline step carries @traceable, so LangSmith shows relevance, rewrite, generation and grounding calls when a key is configured. 4. app.py keeps chat history in st.session_state.messages, shows retrieved sources with excerpts and the Self-RAG notes in expanders, and wraps both index loading and answering in try/except. 5. The code is split cleanly into ingest.py, rag_pipeline.py and app.py, with a requirements.txt, a .gitignore for .env, and JSON-mode outputs for all grading calls.	1. ingest.py:build_vectorstore rebuilds the FAISS index in memory at every start; save it with save_local and load it with load_local unless the data files change. 2. RETRIEVAL_K is 2 with plain similarity search; retrieve 8 to 10 candidates with BM25 hybrid or MMR, then let grade_relevance or a reranker keep the best few. 3. ANSWER_TEMPERATURE is 0.8, which invites drift on amounts and eligibility; set it to 0 or 0.1 for grounded factual answers. 4. The fictional sample_agriculture_schemes.txt is indexed alongside the real document; remove it from data/ so placeholder text can never be retrieved as a scheme answer. 5. The relevance and grounding grades are computed per query but never saved; add an offline question set and log those grades plus hit-rate to a CSV for comparison across settings.	Good	**72**
14/10/2026 15:49:12	rennykenny96@gmail.com	Renny Evangeline S	rennykenny96@gmail.com	https://github.com/rennyevangeline96/tamilnadu-agriculture-rag	1. ingest.py:pdf_to_documents sends pages with images or tables to a GPT-4o-mini vision prompt that rebuilds them as Markdown tables, recovering scheme tables that plain text extraction loses. 2. rag.py:rewrite_query uses the chat history to produce a standalone English search query, so follow-up questions like 'how to apply?' still retrieve the scheme being discussed. 3. The SYSTEM prompt in rag.py injects page-labelled context, restricts answers to that context, says when nothing is found, and returns sorted file-and-page sources after each answer. 4. guardrails.py combines length checks, a NeMo self-check input rail for off-topic or jailbreak prompts, and an output check that retracts answers containing secret-key patterns. 5. api.py exposes /chat and an ndjson /chat/stream endpoint traced with LangSmith, and app.py streams tokens while persisting multiple named conversations to chats.json.	1. eval_rag.py tests a single question; build a 20-question ground-truth set, add contextual recall and hit-rate metrics, and write DeepEval scores to a CSV or JSON report. 2. eval_rag.py re-retrieves context with the raw question instead of the rewritten query used in run_rag; return the actual retrieved docs from run_rag so faithfulness is measured correctly. 3. Chunks only carry source filename and page metadata; add scheme name and department during ingestion and use them for filtering, or add a reranker over a larger candidate set. 4. ingest.py globs 'data/*.pdf' but the committed folder is 'Data/', so re-ingestion finds nothing on Linux; make the input path a configurable absolute path with an existence check. 5. check_nemo_input returns allowed whenever NeMo raises an exception, silently disabling the rail; log the failure and fall back to a cheap keyword or LLM topic check instead.	Good	**73**
14/10/2026 22:30:02	shriram.slm@gmail.com	Shriram K	shriram.slm@gmail.com	https://github.com/shriramk-git/Buildathon_Agri_Schemes_RAG	1. Agriapp.py:retrieve_candidates fuses FAISS similarity and BM25Retriever results with reciprocal rank fusion, and rerank then has the LLM choose the best four of twelve passages. 2. Agriapp.py:answer_question uses an ONLY-evidence system prompt with explicit refusal, [S#] citations and replies in English, Tamil or Tanglish to match the farmer's question. 3. Index.py cleans PyMuPDF text, skips near-empty pages, removes duplicate pages by SHA-256, filters chunks under 60 characters, and writes a manifest.json with PDF hashes beside the saved index. 4. load_resources loads the persisted FAISS index once with st.cache_resource and rebuilds BM25 from its docstore, so no embeddings are recomputed when the app starts. 5. The UI validates question length and script with validate_question, shows source pages with excerpts in an expander, and handles HTTPError separately from other request failures.	1. There is no requirements.txt and Index.py globs PDFs from the repo root; add pinned requirements and move the scheme PDFs into a data/ folder read from an environment variable. 2. load_resources tells users to run 'python build_index.py', which does not exist; change the message to Index.py and compare manifest.json hashes to warn when the index is stale. 3. The app is a single question form with no memory; switch to st.chat_input with st.session_state history so farmers can ask follow-up questions about the same scheme. 4. BM25Retriever uses the default whitespace tokenizer and the splitter has no Tamil-specific separators; add a Unicode-aware tokenizer and Tamil sentence boundaries to both. 5. There is no evaluation code; add a 20-question English/Tamil set with expected source pages and log hit-rate@4 and an LLM-judge faithfulness score to a CSV.	Good	**73**
14/10/2026 22:44:48	jaipsft@gmail.com	Jaikumar S	jaipsft@gmail.com	https://github.com/SJaikumar/AgrischemeAI	1. backend.py:retrieve fuses FAISS and BM25 rankings with reciprocal rank fusion over the original and translated queries, and caps results at two chunks per page. 2. backend.py:upload_documents checks file size and PDF signature, skips duplicates by SHA-256, and persists FAISS plus a manifest and chunks file using atomic JSON writes. 3. pdf_chunks splits each page with 1050/180 RecursiveCharacterTextSplitter settings and stores chunk_id, doc_id, filename, page and part, which lets delete_document remove a PDF's vectors cleanly. 4. backend.py:chat restricts answers to PDF excerpts with [S#] citations, chooses Tamil or English output, and replaces any citation not in the retrieved set with [invalid citation]. 5. frontend.py talks only to the FastAPI backend, supports upload and delete, keeps chat history, and shows each source with RRF score, retrieval hits and a link to the page.	1. There is no evaluation code; add a 20-question Tamil and English test set and measure hit-rate@5, MRR and faithfulness, saving results to a CSV. 2. There is no requirements.txt or .env.example; add both so fastapi, rank-bm25, PyMuPDF and the OpenAI settings are documented and the project installs cleanly. 3. backend.py repeats load_dotenv and BASE_DIR and has a comment glued onto the PDF_DIR line; clean up the config block and remove the debug print statements. 4. AnswerRequest accepts history but chat never uses it and the frontend never sends it; pass recent turns and rewrite follow-up questions before retrieval. 5. Knowledge comes only from user-uploaded PDFs with no OCR; add a loader for official tnagrisnet scheme pages and store department and scheme name as chunk metadata.	Good	**74**
15/10/2026 11:12:17	balajispv1@gmail.com	Balaji.S.P	balajispv1@gmail.com	https://github.com/spbalaji2026/1Buildathon_Tamil-Nadu-Agriculture-Schemes-Assistant.git	1. ingestion/scrape_schemes.py fetch_all_rows pages through the official TN Agrisnet goScheme AJAX API, then is_junk and remove_duplicates drop test rows and keep the richer duplicate. 2. ingestion/build_documents.py split_documents adds a 'Scheme / Department' header to every chunk and keeps scheme_id, department and chunk_index metadata, so no chunk loses its scheme identity. 3. backend/rag_chain.py answer_question uses structured output with an answerable flag and used_scheme_ids, dropping any IDs that were not retrieved, so citations always match real scheme records. 4. backend/validator.py validate_answer runs an independent claim-by-claim LLM judge, and rag_chain falls back to the original scheme text when claims are unsupported instead of showing a hallucinated answer. 5. backend/main.py wraps the chain in FastAPI with request validation and preloads the index, and frontend/app.py shows sources, validation verdicts and the stage where a query stopped.	1. backend/retriever.py does plain dense top-5 search; add BM25 hybrid search or a department metadata filter so short queries like 'tractor subsidy' match scheme names reliably. 2. The test questions exist only in __main__ blocks with printed output; turn them into a labelled eval set with expected scheme_ids and log hit-rate@5 and validator pass rates. 3. frontend/app.py is single-shot Q&A with no chat history; keep turns in st.session_state and pass recent turns to the API so follow-up questions are understood. 4. Nothing handles Tamil questions; detect Tamil input, translate it for retrieval, and ask gpt-4o-mini to reply in the user's language, since farmers will ask in Tamil. 5. requirements.txt is unpinned and exploration scripts like inspect_page.py and find_data_source.py sit in ingestion/; pin versions and move the one-off scripts into a scripts/ folder.	Good	**74**
14/10/2026 13:02:09	jrajkumar09@gmail.com	Rajkumar Jayabal	jrajkumar09@gmail.com	https://github.com/Rajkumar-mechanik/uzhavar-nalan-ai	1. rag.py MultilingualEmbeddings wraps multilingual-e5-base with the required 'query:' and 'passage:' prefixes and normalised vectors, which suits mixed Tamil and English scheme documents. 2. SchemeAssistant uses MMR retrieval with k from the TOP_K env variable and fetch_k four times larger, reducing near-duplicate chunks from the same page in the context. 3. SYSTEM_PROMPT forbids invented amounts, gives a polite fallback to the extension centre, fixes the answer language, and format_docs labels each chunk with its file and page. 4. app.py with chat_store.py saves each conversation as JSON, supports history search and reopening, sends recent turns to the LLM, and shows a bilingual error message on failures. 5. The code is cleanly split into ingest.py, rag.py, chat_store.py and app.py with env-driven model settings and a requirements.txt covering every dependency, so it is easy to run.	1. There is no evaluation code; add a 20-question ground-truth set over the four PDFs and measure hit-rate@5 plus RAGAS faithfulness, saving results to a CSV. 2. Retrieval is dense-only MMR; add BM25 over the same chunks and fuse with EnsembleRetriever or RRF so exact scheme names and amounts are matched reliably. 3. ingest.py stores only file name and language as metadata; extract scheme name and department per chunk so the retriever can filter, and actually use the stored language field. 4. The splitter uses '।', the Devanagari danda, which never appears in Tamil text; replace it with Tamil-appropriate separators such as full stops and line breaks. 5. PyMuPDFLoader returns no text for scanned Tamil pages and ingest.py silently drops them; add an OCR fallback such as Tesseract tam+eng when a page has little text.	Strong	**75**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 23:50:17	bvigneshwaran85@gmail.c	Vigneshwaran B	bvigneshwaran85@gmail.com	https://github.com/Vigneshwaranb-git/Buildathon-RAG-TN-Agri-Scheme-ChatBot	1. app.py:SYSTEM_PROMPT requires Tamil answers, bans outside knowledge, gives an exact refusal sentence, and makes every fact cite the [S1]-style labels that _answer_question attaches to each chunk. 2. rag_core.py:make_embeddings uses the paraphrase-multilingual-MiniLM-L12-v2 model locally, so Tamil questions and English PDF text share one embedding space without extra API cost. 3. build_index writes FAISS plus a manifest to a staging folder and swaps it in atomically, and index_is_current compares a file and settings signature to detect stale indexes. 4. rag_core.py:load_pdf_pages records 1-based page numbers and collects per-file failures and scanned-page warnings, so one bad PDF does not stop the whole indexing run. 5. config.py:Settings.from_env validates chunk size, overlap, top_k and upload limits from the environment, and tests/test_rag_core.py checks metadata preservation and signature changes.	1. The repo has no scheme data or collection code, and scheme_pdfs/ is gitignored; add a script that downloads the official TN scheme PDFs and portal pages. 2. Chunks keep only file name and page metadata; parse scheme name and department from headings during load_pdf_pages so answers and filters can work at the scheme level. 3. Retrieval is a plain similarity_search_with_score with top_k; add BM25 hybrid fusion or MMR and use the returned distance to drop weak matches before generation. 4. The only tests cover chunking and signatures; add an evaluation script with ground-truth Tamil questions that measures hit-rate@k and faithfulness and saves the results. 5. The answer is returned in one block after llm.invoke; stream tokens with llm.stream and st.write_stream so long Tamil answers start appearing sooner.	Strong	**75**
14/10/2026 16:48:19	ramamoorthyprabhuraman	Prabhuraman Ramar	prabhuraman.ramamoorthy@outlook.com	https://github.com/ramamoorthyprabhuraman/tn_agri_rag	1. ingest.py process_pdf detects scanned pages by text length, OCRs them with GPT-4o vision using retries and a model fallback, then falls back to local Tesseract tam+eng. 2. OCR output is cached per page in ocr_output and failed pages are reported at the end, so reruns are cheap and missing content is visible rather than silent. 3. process_pdf also follows links inside the PDFs, scraping pages with trafilatura, fetching YouTube transcripts and transcribing audio with Whisper, each tagged with source, page and type. 4. app.py query_variants translates each question into Tamil and English with the LLM and searches FAISS with all three variants, deduplicating results for cross-lingual retrieval over bge-m3. 5. The system prompt requires context-only answers in the question's language, exact figures with source and page, and an exact bilingual NOT_FOUND message when the answer is missing.	1. There is no requirements.txt and the source PDFs sit only inside data.zip; add a requirements file and a setup step that unzips the data before running ingest.py. 2. Chat history is shown in the UI but only the latest question reaches the LLM; pass recent turns and rewrite follow-ups into standalone queries before retrieval. 3. The retrieval loop and streaming completion in app.py have no try/except, so an API error crashes the page; wrap them and show a friendly error message. 4. check_index.py only prints results for three hardcoded queries; build a ground-truth question set with expected source pages and compute hit-rate@5, MRR and LLM-judged faithfulness. 5. Retrieval is plain similarity search per variant; add BM25 over the chunks fused with RRF, or a multilingual reranker, so exact GO numbers and amounts rank first.	Strong	**77**
14/10/2026 21:13:07	kashok243@gmail.com	Ashok K	ashokpunul@outlook.com	https://github.com/ashokpunul/TN_Agriculture_scheme_RAG_Vector	1. app1.py:hybrid_retrieve fuses FAISS similarity results with a hand-written bm25_search using reciprocal rank fusion, so exact scheme terms and paraphrased questions both surface matching chunks. 2. app1.py:corrective_retrieve reranks candidates with an LLM 0-100 relevance judge and, below the threshold, rewrites the query and retries hybrid retrieval, all toggleable from the sidebar. 3. The PDF ingestion loop extracts tables with page.find_tables, falls back to pytesseract OCR on low-text pages, and attaches detected headings as section_path metadata on every chunk. 4. split_documents_configurable offers recursive, character, token, semantic and parent-child strategies, and splits table documents at row boundaries while repeating the header row in each chunk. 5. rebuild_uploaded_index updates the persisted FAISS index incrementally per source hash and saves it atomically, while read_webpage_content blocks private IPs and redirects via validate_public_http_url.	1. The sources list is built and stored in session_state but never rendered; display it under each answer and label each context chunk as [file, page] so the LLM can cite. 2. The chat prompt joins chunks with a literal "\n\n' string and no source labels; join with real newlines and prefix each chunk with its source and page number. 3. There is no evaluation code and passage_match is never called; build a 20-question TN-scheme ground-truth set and log hit-rate@5 and MRR for hybrid versus FAISS-only retrieval. 4. app1.py is a 2,885-line monolith with a hardcoded Windows tesseract path and no requirements.txt; split it into ingest, retrieval and UI modules and move paths to environment config. 5. all-MiniLM-L6-v2 is English-only and _rag_tokens keeps only [a-zA-Z0-9] tokens, so Tamil text is dropped; switch to multilingual-e5 and a Unicode-aware tokenizer for BM25.	Strong	**77**
14/10/2026 15:53:43	shamsuiux@gmail.com	Samsudeen N	shamsuiux@gmail.com	https://github.com/makesenz/tn-agriculture-schemes	1. app/ingest.py loads three scheme PDFs with PyMuPDFLoader plus the TNAU schemes webpage, adds source, source_type and one-based page_number metadata, and writes a per-source ingestion report. 2. app/rag.py:create_search_query uses the LLM to turn Tamil or Thanglish questions into an English search query, falling back to the original question if the call fails. 3. retrieve_documents over-fetches candidates with similarity_search_with_score and then asks the LLM to select only directly relevant chunk IDs as JSON, returning nothing if parsing fails. 4. ask_question builds context with [Source: file, page N] labels and returns deduplicated sources, while the prompt forbids invented subsidy amounts and requires answering in the user's language. 5. app/test_rag.py runs four regression cases checking required amounts, expected source page, forbidden claims and refusal phrases, and app/streamlit_app.py:ask_backend handles connection, timeout and HTTP errors separately.	1. Retrieval is dense-only with an extra LLM call as filter; add BM25 hybrid search or a cross-encoder reranker so exact scheme names score well without a second paid model call. 2. The answer prompt hardcodes oilseeds eligibility wording and maize-versus-oilseeds rules that match the test cases; remove these and fix wrong answers through better chunks or retrieval instead. 3. test_rag.py only prints pass or fail for four keyword checks; expand to at least 20 questions, add hit-rate and faithfulness scoring, and save each run's results to a CSV. 4. app/rag.py loads FAISS and creates the LLM at import time, and ingest.py uses CWD-relative paths; move setup into a cached init function and resolve paths from PROJECT_ROOT. 5. The backend sends only the latest question, so multi-conversation history in Streamlit never reaches the model; pass recent turns to /api/chat and rewrite follow-up questions using them.	Strong	**78**
15/10/2026 06:14:02	sungaly@gmail.com	Kalyanasundaram Ra	sungaly@gmail.com	https://github.com/kalyanasundaramr/tnschememulti_rag	1. retrieval/hybrid_retriever.py fuses FAISS and BM25 rankings with reciprocal rank fusion (k=60), so exact scheme names and paraphrased questions both surface the right chunk. 2. search_hybrid first runs translate_query_to_english, so Tamil questions are retrieved against the English corpus, and tests/test_retrieval_quality.py checks this with a Tamil query. 3. The pipeline is cleanly split into ingestion, documents, vectorstore, retrieval, generation and ui packages, each a small script with one job and explicit file paths. 4. documents/chunk_schemes.py chunks at 800/120 with explicit separators and add_start_index, and keeps scheme_name and source_url on every chunk through to the FAISS metadata. 5. generation/prompt_template.py restricts answers to the retrieved context, gives a fixed refusal sentence, requires the scheme name and source URL per claim, and mirrors the user's language.	1. ingestion/crawl_schemes.py crawls tnrd.tn.gov.in rural development schemes, not agriculture; point it at tnagrisnet or Uzhavan scheme pages to match the Buildathon brief. 2. vector_retriever.load_vectorstore and keyword_retriever.search_keywords reload FAISS, the embedding model and rebuild BM25 on every query; build them once and cache with st.cache_resource. 3. ui/app.py is a single text area with no chat history; switch to st.chat_message with session_state history and pass recent turns into query translation for follow-ups. 4. graph/build_graph.py links schemes only to a fixed list of ten infrastructure keywords; extract entities such as beneficiaries, departments and benefits with an LLM to make the graph useful. 5. Evaluation is three retrieval asserts on one scheme; add a labelled 20-question set and compute hit-rate@5, MRR and RAGAS faithfulness, logging results to data/evaluation.	Strong	**78**
14/10/2026 13:04:59	gansgpm@gmail.com	GANAPATHYRAMAN	gansgpm@gmail.com	https://github.com/gansgpm/agri-knowledge-assistant.git	1. app/config.py keeps every tunable (models, chunk size, top_k, max_distance, guardrail toggles) in a typed pydantic Settings class read from env, so nothing is hardcoded or secret. 2. app/retrieval.py QAService.answer numbers each passage, demands [n] citations, returns INSUFFICIENT_CONTEXT when unsupported, and maps cited numbers back to Citation objects with source, page and snippet. 3. app/guardrails.py check_answer strips citations that point to no retrieved passage and withholds answers with zero citations, a real post-generation grounding check rather than just a prompt instruction. 4. app/vector_store.py persists FAISS with a manifest recording the embedding model, locks the index on a model mismatch, detects duplicate sources by content hash and saves through a temp-dir swap. 5. app/loaders.py load_url validates URLs against private and loopback addresses, enforces size limits and keeps page and title metadata, while ui/streamlit_app.py adds chat history and clear API error messages.	1. QAService.retrieve calls only similarity_search_with_score; add BM25 over the same chunks fused with reciprocal rank fusion, or a cross-encoder reranker, so exact scheme names rank reliably. 2. The repo ships no Tamil Nadu scheme data or scraper; add an ingestion script for TN Agrisnet scheme pages and store scheme_name and department in chunk metadata for filtering. 3. The tests in tests/ use fake LLMs and embeddings to check behaviour only; add a 20–30 question ground-truth set and report hit-rate@k, MRR and faithfulness to a results CSV. 4. Nothing handles Tamil; detect Tamil questions, translate them for retrieval or use a multilingual embedding model, and tell the model to answer in the user's language. 5. split_documents uses a generic RecursiveCharacterTextSplitter; for scheme documents, split on headings like Eligibility, Benefits and Documents Required so each chunk holds one complete section.	Strong	**79**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 13:38:06	naveensabari1989@gmail.c	Naveensabarinath La	naveensabari1989@gmail.com	Project : https://github.com/naveensabarinath-ai/Buildathon Test Results in PDF : https://github.com/naveensabarinath-ai/Buildathon/tree/main/output	1. pipeline/scrape_tn_agriculture_schemes.py drives the tnagrisnet portal with Playwright, parses listing and modal tables, and saves progress plus a per-scheme status report after every scheme. 2. build_faiss_index.py:scheme_to_documents turns each listing field into its own document with scheme_name, scheme_id and content_type metadata, and split_selectively keeps eligibility and document checklists unsplit. 3. agriculture_rag.py:select_evidence resolves an explicitly named scheme or ID and, for eligibility or document questions, fetches every matching chunk via exact_documents instead of a lossy top-k. 4. SYSTEM_PROMPT in agriculture_rag.py forces answers only from numbered excerpts with [S1] citations, treats sources as untrusted, refuses when evidence is missing, and replies in the question's language. 5. apps/chatbot_api.py wraps the pipeline in FastAPI with validation, LangSmith traceable spans and SQLite token/cost logging, while chatbot_app.py adds chat history, scheme filter and a sources expander.	1. There is no evaluation code at all; add a 25-question ground-truth set, compute hit-rate@5 and MRR for retrieval, run RAGAS faithfulness, and save results to CSV. 2. When no scheme name resolves, fetch_documents falls back to pure dense MMR; add BM25 hybrid search or a cross-encoder reranker so keyword-heavy scheme queries rank correctly. 3. Modules are loaded with importlib.spec_from_file_location in agriculture_rag.py, chatbot_api.py and admin_dashboard.py; use normal package imports from pipeline so tooling, tests and refactoring work reliably. 4. The scraper docstring contains a Windows path 'pipeline\scrape_...' that triggers an invalid-escape SyntaxWarning; make the docstring raw or use forward slashes to keep output clean. 5. Scheme PDFs (GO and guidelines) are only stored as link text and never embedded; download and parse them so detailed subsidy rates and procedures become retrievable.	Strong	**80**
14/10/2026 13:55:29	cbright242@gmail.com	Bright A	cbright242@gmail.com	https://github.com/BrigthA/Agriculture_RAG_Bot	1. retrieve() runs dense_candidates (FAISS) and sparse_candidates (BM25) and merges them with reciprocal_rank_fusion, so exact scheme names and paraphrased questions both surface the right chunks. 2. bm25_tokens uses a Tamil-aware Unicode regex that keeps Tamil words and decimal amounts intact, so keyword search works for Tamil text and rupee figures. 3. index_chunks saves the FAISS index with an embedding_model.txt marker, and _validate_index_model blocks searches when the configured embedding model no longer matches the stored index. 4. safe_get and valid_url only allow listed government domains, refuse private IPs and re-check every redirect, so crawl_approved_links cannot be tricked into fetching internal addresses. 5. The answer prompt restricts facts to the context, tells the model to ignore instructions inside retrieved text, asks it to admit insufficient evidence, and requires [Source n] citations.	1. There is no evaluation code, only thumbs-up feedback rows; add a 20-question ground-truth set and measure hit-rate@6, MRR and LLM-judged faithfulness, saving results to CSV. 2. chat_page calls retrieve() and then stream_agri_answer calls retrieve() again on the same question; return the documents from one call and reuse them for both the citations and the context. 3. Chat history is stored in session_state but never sent to the prompt, so follow-ups like 'how do I apply for it' fail; add a history-aware rewrite step before retrieval. 4. Everything sits in one 1,400-line AgricultureAI_Bot.py; split it into ingestion, retrieval, generation and UI modules so each part can be unit-tested. 5. No corpus or build script is committed, so the index stays empty until an admin uploads files; add a seed script that crawls the default domains and builds it.	Strong	**82**
15/10/2026 00:57:17	iamsuresh.architect@gmail	Suresh Ramalingam	iamsuresh.architect@gmail.com	https://github.com/iamsuresharchitect/TN-Agri-Scheme-Assistant	1. agri_rag/pdf_extractor.py:extract_pdf OCRs scanned pages at 300 DPI with Tamil+English Tesseract, keeps tables as markdown, OCRs embedded images and collects links and file attachments. 2. agri_rag/pipeline.py:_process_pdf follows links found in each PDF, ingests linked web pages and PDFs, transcribes audio and video attachments, and logs unreachable URLs to data/failed_links.csv. 3. agri_rag/chunker.py:chunk_documents splits on paragraph and sentence boundaries with overlap but keeps whole tables together, and vectorstore.py prefixes each chunk with its document title before embedding with bge-m3. 4. agri_rag/rag.py:_pipeline returns a fixed not-found message without calling Claude when no chunk passes SIM_THRESHOLD, and the system prompt requires [n] citations and replies in the detected language. 5. agri_rag/tracing.py wraps retrieval and the Claude client for LangSmith tracing, and app.py sends thumbs-up or thumbs-down ratings to each traced run through send_feedback.	1. There is no evaluation code; add a script with 20 or more ground-truth questions that measures retrieval hit-rate@k and answer faithfulness and saves the results to CSV. 2. FaissStore.search is dense-only cosine search; add BM25 over chunks.jsonl fused with the FAISS results, or a cross-encoder reranker, so exact scheme names and figures rank higher. 3. Chunk metadata has source, page and content_type but no scheme name or department; extract those fields during ingestion so the UI can offer scheme or department filters. 4. url_scraper.py:_get retries failed requests with verify=False whenever SSL_FALLBACK is on, which is the default; set the default to off and log which sources were fetched without certificate checks. 5. The follow-up handling only joins the previous question when the query is under 60 characters; use an LLM rewrite into a standalone question so longer follow-ups also retrieve correctly.	Strong	**83**
16/10/2026 00:15:08	bvigneshwaran85@gmail.c	Vigneshwaran B	bvigneshwaran85@gmail.com	https://github.com/Vigneshwaranb-git/KnowledgeGraph-RAG	1. knowledge_graph.py:build_knowledge_graph uses structured LLM extraction (SchemeExtraction) to write Scheme nodes and typed relations like HAS_ELIGIBILITY and REQUIRES_DOCUMENT into Neo4j with page provenance. 2. app.py:_answer_question merges FAISS chunks with graph facts from retrieve_graph_facts, linking them through the retrieved source files so graph context matches the PDFs actually retrieved. 3. guardrails.py:validate_output rejects answers that cite [S#] IDs that were never retrieved, then the app retries once and runs review_output_grounding as an LLM judge that blocks unsupported answers. 4. rag_core.py:build_index writes a manifest with a corpus/config signature and swaps the index in atomically, so the FAISS index is rebuilt only when PDFs, model or chunk settings change. 5. config.py:Settings.from_env reads every setting (models, chunk size, TOP_K, Neo4j credentials) from environment variables with range validation, and .gitignore keeps .env and indexes out of the repo.	1. The tests folder only checks guardrail regexes and key helpers; add a ground-truth question set and measure hit-rate@k and faithfulness for vector-only versus vector-plus-graph retrieval. 2. Dense retrieval is a plain similarity_search; add BM25 hybrid search or a cross-encoder reranker so exact scheme names and amounts rank reliably above loosely related chunks. 3. Chunks only carry source, file_name and page metadata; copy the extracted scheme_name and department onto each chunk so retrieval can filter by scheme or department. 4. validate_output rejects any Tamil-script answer and the prompt forces English, so Tamil-speaking farmers cannot get Tamil replies; detect the query language and allow grounded Tamil answers. 5. retrieve_graph_facts matches query words with CONTAINS on names and descriptions, which misses synonyms; add a Neo4j vector index on entity descriptions or link entities using embeddings.	Strong	**83**
14/10/2026 17:28:28	prakashvna@gmail.com	Prakash Pa	prakashvna@gmail.com	https://github.com/deeplyhumanlearning/uzhavar-gpt	1. scrape_schemes.py calls the TNAGRISNET scheme API for eight departments and scrapes tn.gov.in, writing bilingual English and Tamil scheme sections tagged with department and source URL. 2. ingest.py:load_pdf drops mojibake and legacy-font Tamil lines from the ICAR PDFs, and load_markdown makes one Document per '## scheme' section so chunks never mix two schemes. 3. rag.py:SchemeRAG.retrieve rewrites the question with chat history into a standalone English query, searches FAISS with both the rewritten and original question, and removes duplicate results. 4. ANSWER_PROMPT tells the model to answer only from labelled context, quote amounts exactly, reply in the detected language, and point to the Kisan Call Centre when the answer is missing. 5. api.py loads the persisted FAISS index once at startup and streams NDJSON with sources, while app.py adds multi-chat history, citations and Tamil voice input and output through voice.py.	1. There is no evaluation code at all; add an eval.py with about 20 Tamil and English questions mapped to expected sources, and log hit-rate, MRR and LLM-judge faithfulness to CSV. 2. Retrieval is pure dense similarity_search, so acronyms like PMKSY or exact GO numbers can be missed; add BM25 over the same chunks and fuse the rankings with reciprocal rank fusion. 3. The scheme and department fields are stored in metadata but never used as filters; detect the department or scheme in the rewritten query and pass a FAISS metadata filter. 4. Every question makes an extra LLM rewrite call, even on a first English turn; skip the rewrite when there is no history and no Tamil, to save latency. 5. There are no automated tests for ingest.py's line-cleaning or markdown splitting; add pytest cases for is_clean_line and load_markdown so data-refresh regressions are caught early.	Strong	**84**
14/10/2026 23:01:48	fernandesreegan@gmail.co	Fernandes Reegan A	fernandesreegan@gmail.com	https://github.com/fernandesreegan/RAG-Uzhavan-AI	1. uzhavan/answers.py:grounded_claims drops any claim whose citation quote is not an exact substring of the retrieved evidence, then a second LLM verifier pass rejects unsupported claims. 2. uzhavan/loaders.py:parse_file handles PDF layout text with eng+tam Tesseract OCR fallback, plus XLSX, CSV, DOCX and HTML tables, storing page numbers and extraction method as metadata. 3. vectorstore.py:build_index writes versioned FAISS folders with a JSON docstore and sha256 manifest, and load_index refuses to load if hashes, model or dimensions do not match. 4. vectorstore.py:retrieve oversamples candidates, converts L2 distance to cosine similarity, applies a minimum-score threshold and department filter, so weak matches trigger abstention before the LLM runs. 5. uzhavan/crawler.py enforces exact host allowlists, robots.txt rules and per-seed caps, while answers.py:abstain returns a proper Tamil or English refusal when evidence is missing.	1. tests/test_core.py only unit-tests parsing and logic with mocked models; add a ground-truth question set and measure retrieval hit-rate@k, MRR and answer faithfulness, saving reports. 2. Retrieval is dense-only even though candidates are oversampled; add BM25 hybrid fusion or a cross-encoder reranker over those 60 candidates to improve ranking for exact scheme names. 3. The splitter separator '\u0be4' in make_chunks is an unassigned code point, so it never matches; drop it or split Tamil text on sentence punctuation that actually appears in the documents. 4. Follow-up handling just concatenates the previous question into the retrieval query; use an LLM-based standalone query rewrite so pronoun follow-ups retrieve the right scheme reliably. 5. The app cannot answer until manage.py crawl and build have run; ship a small prebuilt sample index or a first-run build button with clear status to simplify setup.	Strong	**84**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 12:58:18	suhail.beststar@gmail.com	Fathima Suhaila	Suhail.beststar@gmail.com	https://github.com/strangefirm/TN-AgriAssist-Multi-lingual-Hybrid-RAG-Evaluation-System-for-State-Subsidies	1. RagEngine.retrieve fuses FAISS IndexFlatIP and BM25Okapi rankings with weighted_rrf, then reranks the top 20 with a cross-encoder down to three passages before generation. 2. _load_or_build_index fingerprints source files, embedding model and chunk settings with SHA-256, so the persisted index_store is reused and only rebuilt when something actually changes. 3. SYSTEM_PROMPT demands per-claim [n] citations and an exact abstention message, and verify_answer flags invalid citation numbers and figures not found word-for-word in the retrieved context. 4. build_search_question rewrites short or pronoun-based follow-ups by adding the previous question and a trimmed previous answer, so retrieval keeps the scheme context across turns. 5. evaluate_in_background runs DeepEval faithfulness and answer-relevancy on each non-abstained answer in a thread and appends scores to logs/eval_log.jsonl without slowing the chat.	1. Ingestion only reads agri25_26.pdf with pypdf and keeps just source and page; add scheme name and department metadata and more official sources such as tnagrisnet scheme pages. 2. The reranker cross-encoder/ms-marco-MiniLM-L-6-v2 is English-only and scores Tamil questions poorly; swap in a multilingual reranker such as BAAI/bge-reranker-v2-m3. 3. Evaluation is online only and 13 of 59 log entries are errors; add an offline ground-truth question set measuring hit-rate@3 and MRR, reported as a summary table. 4. SYSTEM_PROMPT answers in Tamil only when the user explicitly asks, so Tamil questions get English replies; detect Tamil script in the question and answer in that language. 5. app1.py is an 837-line single file with models, weights and k hardcoded as constants; split it into ingest, retrieval and UI modules and read settings from environment variables.	Strong	**85**
14/10/2026 16:13:43	mohamedfaiyaz1997@gma	Mohamed Faiyaz	mohamedfaiyaz1997@gmail.com	https://github.com/MohamedfaiyazO5/GenAI_SE_Class/tree/main/Buildathon_Week2	1. src/crag.py builds a real LangGraph corrective-RAG graph: retrieve, grade each chunk with the LLM, then route to a DuckDuckGo web-search fallback only when every chunk is rejected. 2. src/extractor.py:process_pdf extracts page text in 1200/150 chunks, pulls tables with PyMuPDF find_tables, and sends image-heavy pages to GPT-4o-mini vision so chart values become searchable. 3. vector_store.py:similarity_search merges FAISS results with an IDF-weighted keyword pass that has difflib typo correction and a 'tamilnadu' alias, so misspelled or joined scheme terms still match. 4. VectorStoreManager keeps a processed_files.json registry and persisted FAISS index, embeds only new PDFs, and rebuild_index swaps in a staged index only after ingestion succeeds. 5. The _generate prompt labels chunks as [file, p. N] and requests citations in the user's language, while backend.py streams tokens over SSE with persistent session memory.	1. There is no RAG quality evaluation, only unit tests with fakes; turn static/sample_question_test.txt into a ground-truth set and log hit-rate@8, MRR and RAGAS faithfulness to a CSV. 2. The generation prompt asks for clarification but has no clear 'not in the documents' answer; add an explicit refusal and flag answers that come from web fallback. 3. The hand-written keyword scorer always ranks keyword hits ahead of semantic hits; replace it with BM25Retriever plus reciprocal rank fusion, or add a cross-encoder reranker after merging. 4. _grade_documents makes eight separate LLM calls one after another for every question; grade all chunks in one structured-output call or run them in parallel to cut latency and cost. 5. multimodal_crag_app/requirements.txt is empty and tests/test_rag.py is blank; move the dependencies into the app folder and stop committing chat_memory/, feedback/ and vector_index_test/.	Strong	**86**
14/10/2026 22:56:55	arunkumar15mca002@gma	Arunkumar Ayyanar	arunkumar15mca002@gmail.com	https://github.com/ArunAIForge/UzhavanAI	1. rag_engine.py AgriRAGEngine.retrieve merges FAISS dense hits with BM25Okapi keyword hits, so exact scheme names and paraphrased Tamil questions can both reach the candidate pool. 2. The merged candidates are reranked with FlashRank ms-marco-TinyBERT, and main.py execute_rag drops anything below RAG_MIN_RERANK_SCORE, refusing with a fixed Tamil message instead of guessing. 3. main.py execute_rag numbers every chunk as [S1], [S2] in the prompt and rejects answers citing IDs that were not supplied, which is real citation validation, not just prompting. 4. ingest.py tags every PDF page, web page and cached text file with scheme_name, doc_type, department and a sha256 fingerprint, so unchanged sources are skipped on re-ingestion. 5. main.py _persist_incremental_documents rebuilds the index in a staging folder and swaps the files in atomically, while app.py streams answers over SSE through backend_client.stream_query_events.	1. There is no ground-truth evaluation; add 20-30 Tamil and English scheme questions with expected sources, then log hit-rate@5, MRR and RAGAS faithfulness to a CSV. 2. tests/test_rag_api_flow.py calls main._validate_grounded_answer, which does not exist in main.py, so that test fails; implement the validator or delete the stale test. 3. main.py stubs tiktoken with MagicMock at import and imports ingest while ingest imports main; remove the hack and move shared code like DirectOpenAIEmbeddings into its own module. 4. app.py is about 1,700 lines and app-bkp.py is a near-copy of it; delete the backup and split UI, styling, TTS and dossier rendering into separate modules. 5. app.py imports gTTS but requirements.txt does not list it, and .gitignore still contains PowerShell here-string text; add gTTS to requirements and clean up .gitignore.	Strong	**86**
14/10/2026 13:28:33	dayanithimadavan@gmail.c	DayanithiMadavan	dayanithimadavan@gmail.com	https://github.com/dayanithi-ai-program/RAG_Government_Scheme_Chatbox	1. extract_schemes.py and download_pdfs.py:download_pdf pull official tn.gov.in scheme PDFs with three retries, a %PDF- signature check and per-file progress saved to JSON. 2. document_loader.py:load_pdf_documents stamps every page with source file, department, scheme name and source URL, so each chunk can be traced back to its official document. 3. clean_chunks.py removes page-number-only, cover, index and unreadable chunks and writes the rejects to removed_chunks.json with a reason, giving an auditable cleaning step. 4. rag_engine.py:rewrite_question uses chat history to resolve follow-ups and translates Tamil questions into English search queries, while PROMPT answers back in the user's own language. 5. evaluation/evaluate.py runs 15 labelled questions through ask_question and scores expected facts, document/page match, abstention and clarification, saving everything to evaluation_results.csv.	1. The department filter never matches because build_vector_store.py stores department_names as a JSON list string; store a plain string and filter on that exact value. 2. knowledge_graph/graph_builder.py and graph_retriever.py are never imported by rag_engine.py or streamlit.py; wire graph retrieval into ask_question or remove the folder. 3. Retrieval is dense MMR only; add BM25 over the same chunks and fuse with reciprocal rank fusion, or add a cross-encoder reranker, so exact scheme names and amounts rank higher. 4. Evaluation uses substring fact matching only; add an LLM-as-judge or RAGAS faithfulness score per answer and report retrieval hit-rate@5 next to the existing CSV metrics. 5. Ten one-off inspect_*, validate_*, retry and test_vector_store.py scripts plus committed __pycache__ folders sit at the root; move pipeline steps into a package with one run script.	Strong	**87**
14/10/2026 16:20:27	ksmurari@gmail.com	Murari Konduru	Ksmurari@yahoo.com	https://github.com/ksmurari1/tn-agri-schemes-rag	1. src/retrieval.py:retrieve_documents fuses FAISS similarity results and BM25 rankings with reciprocal_rank_fusion, and app.py defaults to Hybrid mode, so exact GO numbers and paraphrased questions both match. 2. src/ingestion.py:_pdf_documents extracts PDF text page by page with PyMuPDF and falls back to Tesseract OCR (eng+tam) for near-empty pages, saving page and extraction_method metadata on each document. 3. ingest_content hashes every source with SHA-256, skips identical content through the SQLite registry, and adds new chunks to the persisted FAISS index instead of rebuilding it each run. 4. src/qa.py:SYSTEM_PROMPT requires answers only from retrieved context, inline [title, page] citations, and stated evidence gaps, and answer_question returns structured sources that the UI renders. 5. app.py:process_question runs guardrails.check_question (injection regex plus OpenAI moderation, failing closed) before retrieval, and tests/ plus .github/workflows/ci.yml run pytest on every push.	1. The tests check tokenizer, BM25, RRF and mocked UI flow but never answer quality; add a ground-truth scheme question set and log hit-rate@5, MRR and RAGAS faithfulness to CSV. 2. expand_retrieval_queries uses hand-written Tamil keyword rules with hardcoded English text such as Rs.1200 per quintal; replace them with LLM translation so any Tamil question gets a matching English query. 3. _pdf_documents hardcodes tesseract_cmd to C:\Program Files\Tesseract-OCR\tesseract.exe, which breaks OCR on Linux and Streamlit Cloud; read TESSERACT_CMD from the environment and only set it when present. 4. Chunks carry only source title, URL and page; add department, GO number and year from catalogue rows so the year filter uses structured fields, not filenames. 5. voice.py at the repository root is an exact unused copy of src/voice.py, and app.py is 866 lines; delete the duplicate and split the page renderers into separate modules.	Strong	**89**
14/10/2026 17:12:36	mbalatamil007@gmail.com	Balamanikandan M	mbalatamil007@gmail.com	https://github.com/Mbalatamil007/Buildathon-TN-Agriculture-RAG/tree/main	1. retriever.retrieve runs BM25 and FAISS for each planned query and merges them with weighted reciprocal rank fusion in retrieval_utils.fuse_results, logging which method found each source. 2. retrieval_utils.tokenize_text keeps Unicode combining marks, so BM25 tokenizes Tamil words correctly, and tests/test_retrieval.py verifies this alongside the fusion ranking behaviour. 3. dynamic_user_query.plan_question uses structured LLM output to produce a standalone question plus up to three Tamil or English search queries, resolving follow-ups from recent history. 4. scope_resources builds a district-only FAISS sub-index from stored vectors without re-embedding, giving a real metadata filter driven by the district dropdown in app.py. 5. create_faiss_db.py writes versioned snapshots with SHA-256 checksums and an atomic CURRENT pointer, and load_knowledge refuses to load if chunk settings or hashes no longer match.	1. The committed knowledge PDF comes from hand-curated district_sources.json summaries; live-test the data_ingestion.collect selectors and commit a portal-scraped corpus with GO PDFs. 2. Tests only cover tokenization, fusion and loaders; add a ground-truth Q&A set and score hit-rate@6, MRR and RAGAS faithfulness per district, saving results to a CSV. 3. text_splitter.split_pdf uses the Devanagari danda as a separator, which never occurs in Tamil text; replace it with Tamil-appropriate sentence breaks or rely on newlines. 4. The vector_db snapshot is gitignored, so the app shows an error until create_faiss_db.py is run; build the index automatically on first launch when CURRENT is missing. 5. Retrieval has no reranking step after fusion; add a cross-encoder or LLM reranker over the top fused passages before selecting the six sources sent to the model.	Outstanding	**93**

Timestamp	Email address	Full Name	Email ID	GitHub Repository Link (Paste the public GitHub link . Make sure the repository is Public)	What is good	What needs to improve	Grade	Score /100
14/10/2026 12:58:38	denomelvin24@gmail.com	Deno Melvin Paul D	denomelvin24@gmail.com	https://github.com/Deno-AI-Automation-Lab/GenAI-Projects/tree/main/agri_scheme	1. tnagrischemes.py crawl_website does a robots-aware breadth-first crawl of tnagrisnet with a Playwright fallback, turning the scheme table into labelled per-scheme records with department and GO links. 2. pdf_to_text sends scanned Tamil GO pages to GPT-4o vision with a hash-keyed ocr_cache, and summarize_go adds English KEY FACTS so English questions can find Tamil orders. 3. plan_question uses structured output to rewrite follow-ups, pick named schemes and a department filter, and gather_context then injects full scheme records plus neighbouring GO chunks. 4. agrischemenetworkx.py KnowledgeGraphStore.search fuses vector, BM25, scheme-summary and entity retrievers with reciprocal rank fusion, and run_plan adds read-only LLM-planned graph traversals to the context. 5. run_eval scores generated questions plus guardrail cases with an LLM judge for faithfulness and relevance, writes eval_results.csv, and judge_and_log attaches live scores to LangSmith traces.	1. Three near-parallel apps plus an older Test.py copy sit in one folder; keep one entry point and split ingestion, retrieval and UI into separate modules of a package. 2. The only requirements.txt is the parent course file listing three LangChain packages; add a project requirements file with streamlit, faiss-cpu, rank_bm25, networkx, playwright, reportlab and pymupdf. 3. Evaluation exists only in the KG variants and judges answers, not retrieval; add a hand-labelled question set with expected schemes and report hit-rate@k and MRR for each app. 4. tokens() in agrischemenetworkx.py matches only [a-z0-9], so BM25 and entity search drop Tamil words; extend TOKEN_RX to the Tamil Unicode block as bm25_tokens in tnagrischemes.py does. 5. pick_font only checks Windows font paths and http_get retries with verify=False; bundle a Tamil TTF fallback for Linux and fix certificate handling instead of disabling SSL verification.	Outstanding	**94**
14/10/2026 13:42:30	cvijayvinoth@gmail.com	Vijay Vinoth C	cvijayvinoth@gmail.com	https://www.skool.com/social-eagle-ai-6384/se-ai-b12-2-w2-buildathon-uzhavan-ai-tamil-nadu-agriculture-knowledge-assistant			Not scored	Not scored
14/10/2026 19:39:12	sundaji@gmail.com	SUNDARARAJ R	sundaji@gmail.com	https://github.com/sundaji1969/Buildathon_Agriculture_App			Not scored	Not scored
15/10/2026 02:02:39	priazpage@gmail.com	Priyatharshini Govind	priazpage@gmail.com	https://github.com/prijiturise/buildathon			Not scored	Not scored
15/10/2026 06:18:17	vaibhav229476@gmail.com	vignesh	vaibhav229476@gmail.com	https://github.com/VICCKKY/sourcepilot/blob/48b6ddbf637e4a8b52866acdd570f1ef86e6f4d6/code			Not scored	Not scored